CS3 RESEARCH EXPERIENCE FOR TEACHERS

Object Detection

Lecture notes

Mehmet Kerem Turkcan

Department of Civil Engineering & Engineering Mechanics, Columbia University

PART 1

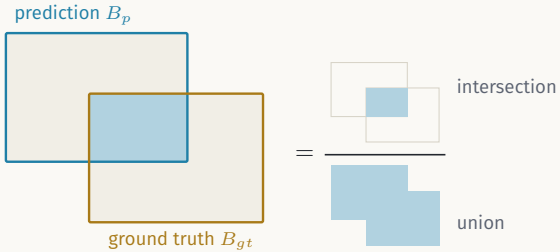
Review I

Intersection over union measures box overlap

- Compare a predicted box B_p with a ground-truth box B_{gt}.
- Divide the area they share by the area they cover together.
- $\text{IoU} = 1$ for a perfect match and 0 for boxes that do not touch.

Definition

$$\text{IoU}(B_p, B_{gt}) = \frac{\text{area}(B_p \cap B_{gt})}{\text{area}(B_p \cup B_{gt})}$$



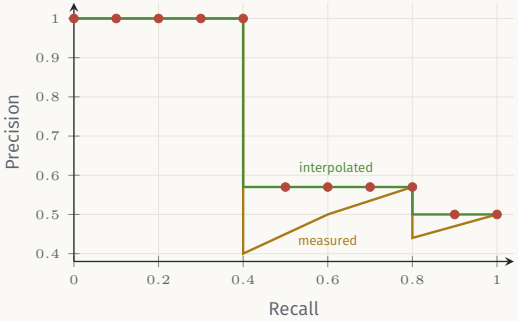
Mean average precision averages over classes

- Mean average precision (mAP) measures how accurately a model detects objects of every class.
- It is the most common metric for evaluating object detection.
- Compute the average precision AP_i of each class i, then average over the N classes.

Mean average precision

$$\mathrm{mAP} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{AP}_i$$

Average precision from an interpolated PR curve



- Pascal VOC evaluation before 2008 **interpolates** the precision–recall curve, for example at 11 recall points.
- Interpolated precision is the best precision at equal or higher recall: $p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$.

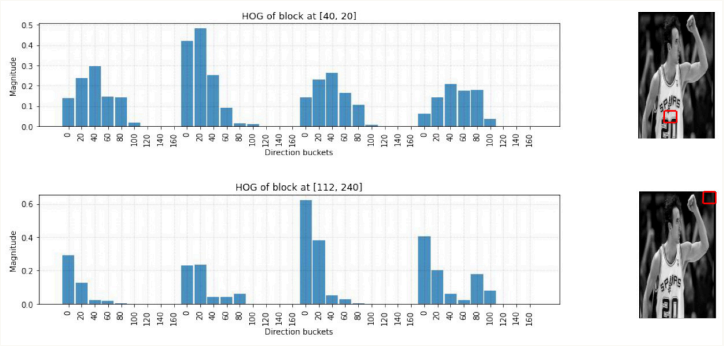
11-point AP

$\text{AP} = \frac{1}{11} \sum_{r \in \{0, 0.1, \ldots, 1\}} p_{\text{interp}}(r)$

Here, $\text{AP} = (5 \times 1.0 + 4 \times 0.57 + 2 \times 0.5)/11 \approx 0.75$.

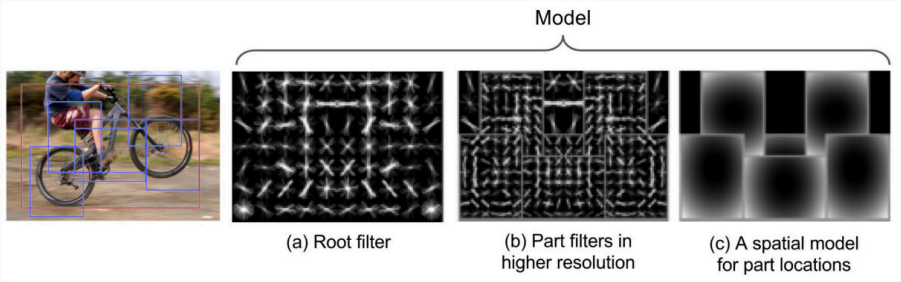
HOG describes blocks by gradient directions

- A histogram of oriented gradients (HOG) counts the gradient directions inside each block of the image.
- The blocks at $[40, 20]$ and $[112, 240]$, marked in red, give clearly different histograms.



The deformable parts model has three components

- A coarse **root filter** defines a detection window that approximately covers the entire object.
- Several **part filters** cover parts of the object and are computed at twice the resolution.
- A **spatial model** scores the locations of the part filters relative to the root.



Felzenszwalb, Girshick, McAllester, and Ramanan, *Object Detection with Discriminatively Trained Part-Based Models*, IEEE TPAMI 2010.

Non-maximum suppression removes duplicate boxes

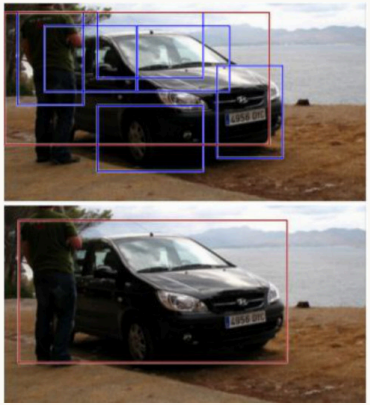
NMS keeps the best-scoring bounding box and suppresses the other boxes that overlap it.

NMS algorithm

Input: boxes $B = \{b_1, \ldots, b_N\}$, scores $S = \{s_1, \ldots, s_N\}$, threshold T

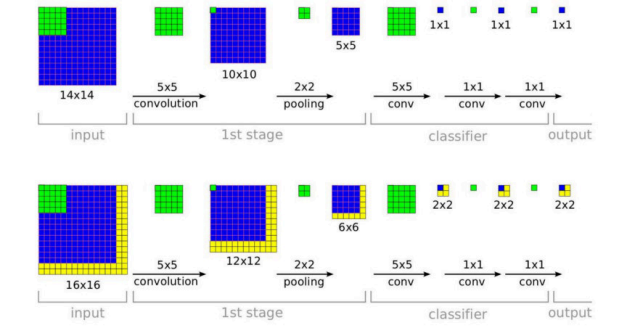
$D \leftarrow \emptyset$
while $B \neq \emptyset$ **do**
  $m \leftarrow \arg\max S$; $M \leftarrow b_m$
  $D \leftarrow D \cup \{M\}$; $B \leftarrow B \setminus \{M\}$
  for $b_i \in B$ **do**
    if $\mathrm{IoU}(M, b_i) \geq T$ **then** $B \leftarrow B \setminus \{b_i\}$; $S \leftarrow S \setminus \{s_i\}$
return D, S

The highlighted step removes the redundant boxes.



Before (top) and after (bottom) NMS.

OverFeat: an almost-free sliding window



Sermanet et al., *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*, ICLR 2014, Fig. 5.

- During training, the ConvNet produces a single spatial output (top).
- At test time, a larger image produces a spatial output map, such as 2×2 (bottom).
- Because every layer is convolutional, the extra computation is limited to the yellow regions.

Evaluating segmentation

Common evaluation methods:

- Pixel-wise accuracy
- Mean accuracy
- Mean IoU: the mean over classes of the region intersection over union
- Frequency-weighted IoU
- BF measure (boundary F1 measure)

Definitions

n_{ij}: pixels of class i predicted as class j; $t_i = \sum_j n_{ij}$; n_{cl}: number of classes.

$$\text{pixel accuracy} = \sum_i n_{ii} / \sum_i t_i$$

$$\text{mean accuracy} = \frac{1}{n_{\text{cl}}} \sum_i n_{ii}/t_i$$

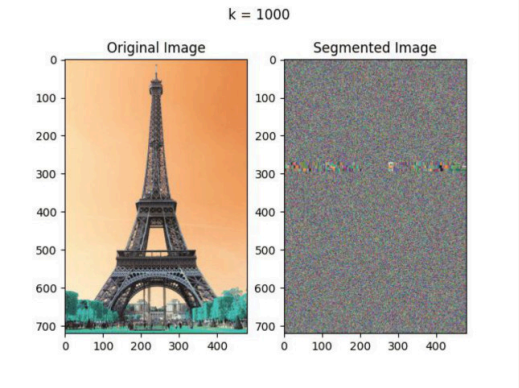
$$\text{mean IoU} = \frac{1}{n_{\text{cl}}} \sum_i \frac{n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}}$$

$$\text{frequency-weighted IoU} = \left(\sum_k t_k\right)^{-1} \sum_i \frac{t_i\, n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}}$$

Definitions as given in Long, Shelhamer, and Darrell, *Fully Convolutional Networks for Semantic Segmentation*, CVPR 2015.

Image segmentation with Felzenszwalb's algorithm

- A graph-based method that groups neighboring pixels into regions.
- The parameter k sets the scale of the result: larger k favors larger regions.
- The example uses $k = 1000$.

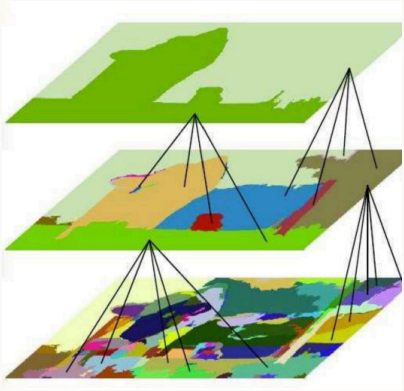


Felzenszwalb and Huttenlocher, *Efficient Graph-Based Image Segmentation*, IJCV 2004.

Selective search merges similar regions bottom-up

Starting from an initial segmentation (bottom layer):

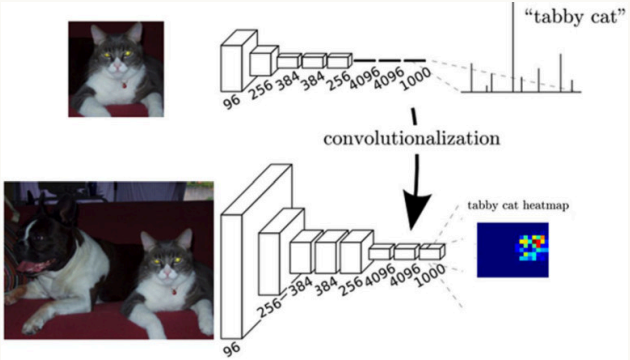
1. Merge the two most similar regions according to a similarity S built from
   - color similarity texture similarity
   - size similarity fill similarity
2. Update the similarities between the new region and its neighbors.
3. Return to step 1 until the whole image is a single region.



Uijlings et al., *Selective Search for Object Recognition*, IJCV 2013.

FCNs turn a classifier into a dense predictor

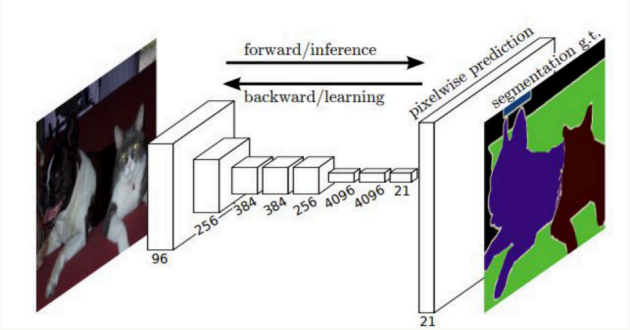
- **Convolutionalization**: the fully connected layers of a classification network are rewritten as convolutions.
- The network then accepts an image of any size and outputs a heatmap for each class, such as “tabby cat”.



Long, Shelhamer, and Darrell, *Fully Convolutional Networks for Semantic Segmentation*, CVPR 2015.

FCNs learn pixelwise prediction end to end

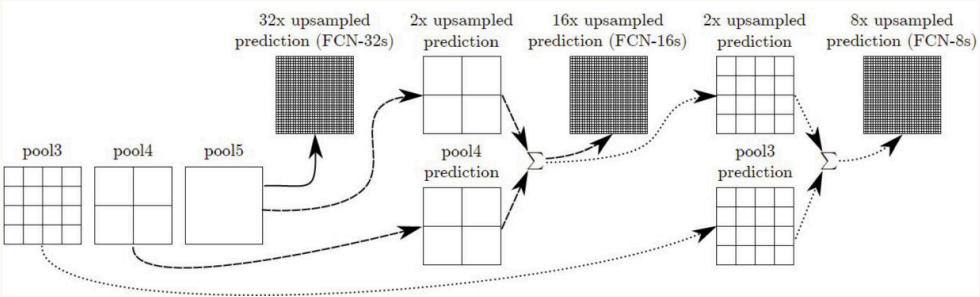
- The forward pass produces a pixelwise prediction with one channel per class (21 for Pascal VOC).
- The loss against the segmentation ground truth is backpropagated through the whole network.



Long, Shelhamer, and Darrell, *Fully Convolutional Networks for Semantic Segmentation*, CVPR 2015.

Skip connections refine coarse FCN predictions

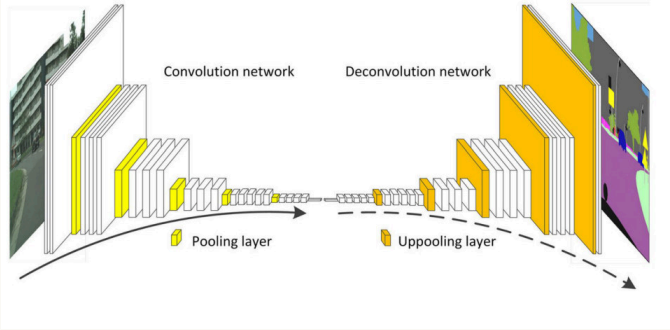
- FCN-32s upsamples the prediction from pool5 by $32\times$ in one step.
- FCN-16s and FCN-8s add predictions from the finer pool4 and pool3 layers before upsampling, recovering more detail.



Long, Shelhamer, and Darrell, *Fully Convolutional Networks for Semantic Segmentation*, CVPR 2015.

Deconvolution networks mirror the encoder

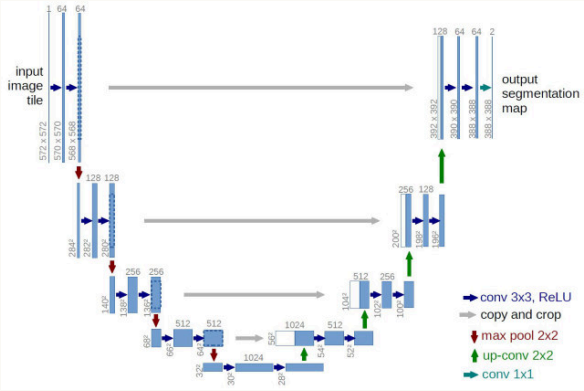
- A convolution network encodes the image, shrinking it with pooling layers.
- A mirrored deconvolution network decodes it with unpooling layers back to a full-resolution segmentation.



Noh, Hong, and Han, *Learning Deconvolution Network for Semantic Segmentation*, ICCV 2015.

U-Net

- A contracting path of 3×3 convolutions and 2×2 max pooling.
- An expanding path of 2×2 up-convolutions.
- Copy-and-crop connections pass encoder features to the decoder at each level.



Ronneberger, Fischer, and Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015.

UNet++ nests dense skip pathways

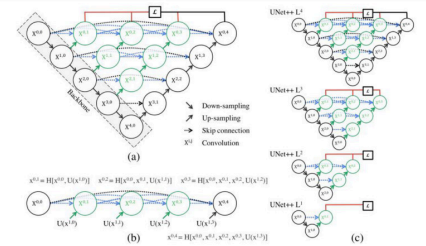
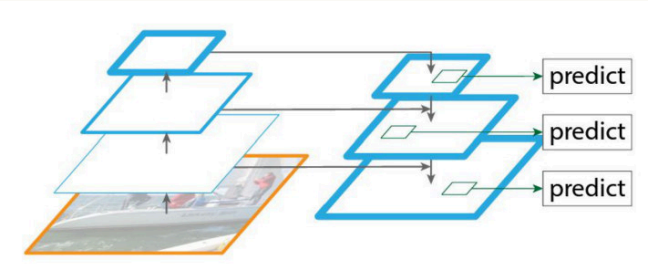


Fig. 1: (a) UNet++ consists of an encoder and decoder that are connected through a series of nested dense convolutional blocks. The main idea behind UNet++ is to bridge the semantic gap between the feature maps of the encoder and decoder prior to fusion. For example, the semantic gap between $(X^{0,0}, X^{1,3})$ is bridged using a dense convolution block with three convolution layers. In the graphical abstract, black indicates the original U-Net, green and blue show dense convolution blocks on the skip pathways, and red indicates deep supervision. Red, green, and blue components distinguish UNet++ from U-Net. (b) Detailed analysis of the first skip pathway of UNet++. (c) UNet++ can be pruned at inference time, if trained with deep supervision.

Zhou et al., *UNet++: A Nested U-Net Architecture for Medical Image Segmentation*, DLMIA 2018.

Feature pyramid networks predict at every scale

- A bottom-up pathway computes features at decreasing resolutions.
- A top-down pathway with lateral connections brings strong semantic features back to each resolution.
- Predictions are made at each level of the pyramid.



Lin et al., *Feature Pyramid Networks for Object Detection*, CVPR 2017.

ReInspect decodes detections with an LSTM

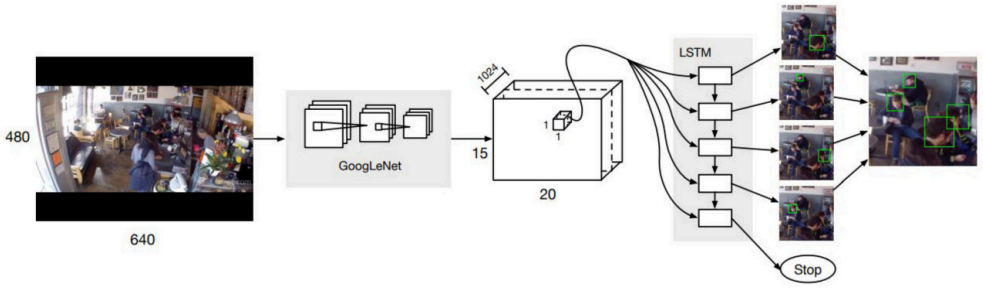
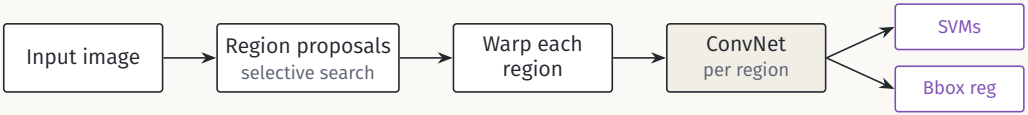


Figure 2: Our system first encodes an image into a block of high level features. An LSTM then acts as a controller, decoding this information into a set of detections.

Stewart, Andriluka, and Ng, *End-to-End People Detection in Crowded Scenes*, CVPR 2016.

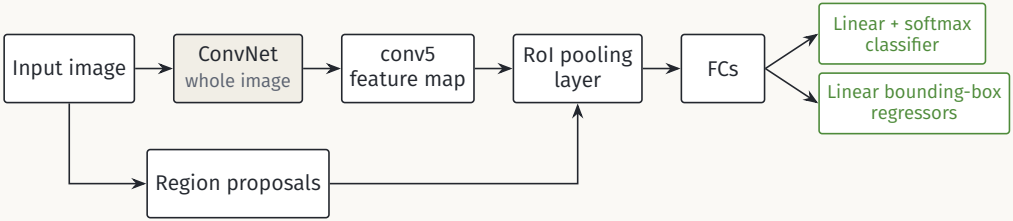
R-CNN runs a ConvNet on every region



- About 2000 region proposals are cropped from the image and warped to a fixed size.
- A ConvNet computes features for each warped region separately.
- Class-specific SVMs score each region.
- Bounding-box regressors refine its coordinates.

Girshick, Donahue, Darrell, and Malik, *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*, CVPR 2014.

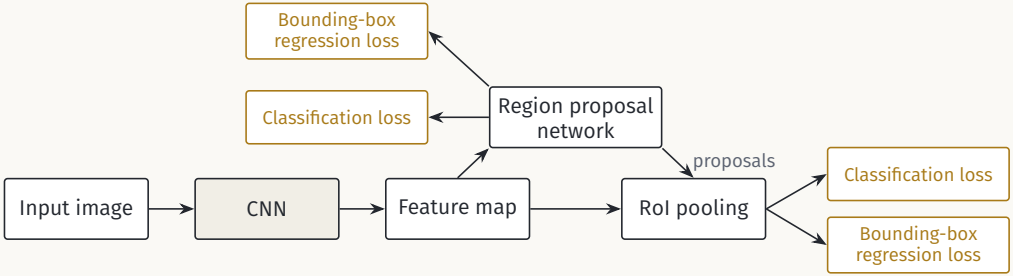
Fast R-CNN runs the ConvNet once per image



- The whole image goes through the ConvNet once, producing a shared “conv5” feature map.
- The RoI pooling layer extracts a fixed-size feature for each region proposal from that map.
- Fully connected layers feed a softmax classifier and bounding-box regressors, trained together.

Girshick, *Fast R-CNN*, ICCV 2015.

Faster R-CNN learns its own proposals



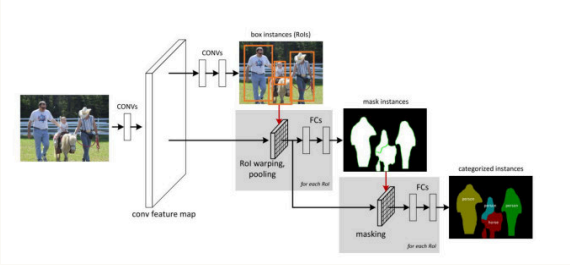
- A region proposal network (RPN) predicts proposals from the shared feature map, trained with its own classification and box-regression losses.
- RoI pooling extracts features for each proposal; the detection head has a second pair of losses.

Ren, He, Girshick, and Sun, *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*, NeurIPS 2015.

Multi-task network cascades

Instance-aware semantic segmentation in three stages

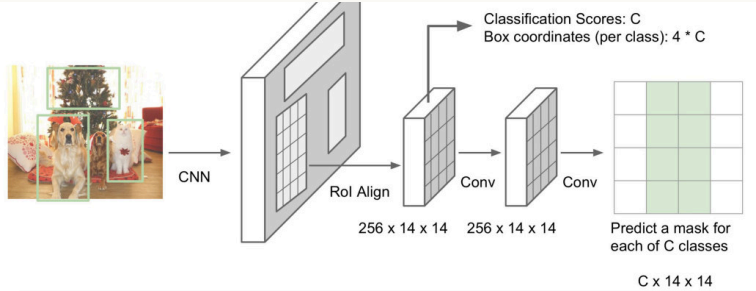
- Three stages share one convolutional feature map: box instances (RoIs), mask instances, and categorized instances.
- Each stage uses the output of the previous one.



Dai, He, and Sun, *Instance-Aware Semantic Segmentation via Multi-Task Network Cascades*, CVPR 2016.

Mask R-CNN adds a mask branch to the detector

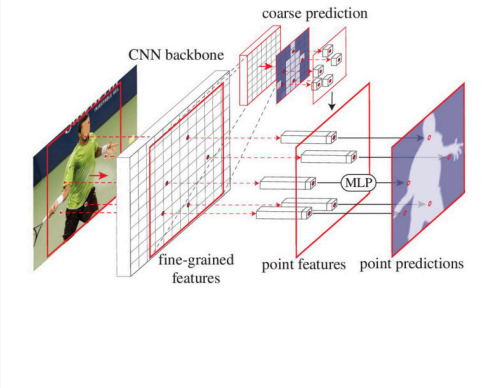
- **RoI Align** extracts a $256 \times 14 \times 14$ feature for each region.
- One branch outputs C classification scores and $4C$ box coordinates (four per class).
- A convolutional branch predicts a 14×14 mask for each of the C classes.



He, Gkioxari, Dollár, and Girshick, *Mask R-CNN*, ICCV 2017.

PointRend refines masks at selected points

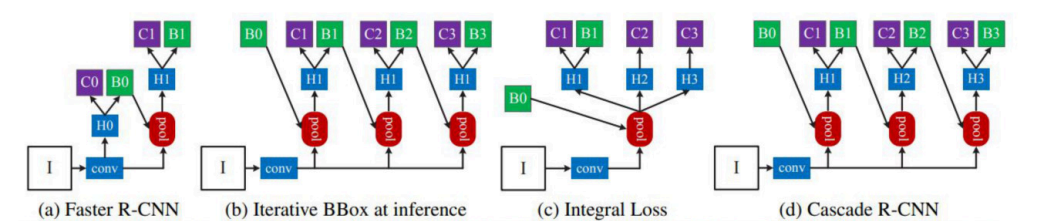
- Start from a coarse mask prediction.
- Select uncertain points and gather fine-grained features from the CNN backbone at those points.
- A small MLP predicts the label of each point.



Kirillov, Wu, He, and Girshick, *PointRend: Image Segmentation as Rendering*, CVPR 2020.

Cascade R-CNN stacks detection heads

- Faster R-CNN uses one head; Cascade R-CNN chains several heads, each refining the boxes of the previous one.



(a) Faster R-CNN (b) Iterative BBox at inference (c) Integral Loss (d) Cascade R-CNN

Figure 3. The architectures of different frameworks. "I" is input image, "conv" backbone convolutions, "pool" region-wise feature extraction, "H" network head, "B" bounding box, and "C" classification. "B0" is proposals in all architectures.

Cai and Vasconcelos, *Cascade R-CNN: Delving into High Quality Object Detection*, CVPR 2018.

PART 2

Single Shot MultiBox Detector (SSD)

Outline

- Fundamentals of object detection
- Evaluation criteria for object detection
- Conventional object detection methods
  - HOG DPM OverFeat
- Segmentation
  - Semantic and instance segmentation
- Evaluation criteria for semantic segmentation
- Early segmentation approaches
- Fully convolutional networks (FCNs)
- U-Net segmentation
- Region-based CNNs (R-CNN)
- Fast R-CNN
- Faster R-CNN
- Multi-task network cascades
- Mask R-CNN instance segmentation
- **Single Shot MultiBox Detector (SSD), a single-stage detector**
- YOLO (You Only Look Once), a single-stage detector
- RetinaNet

SSD is a single-stage detector

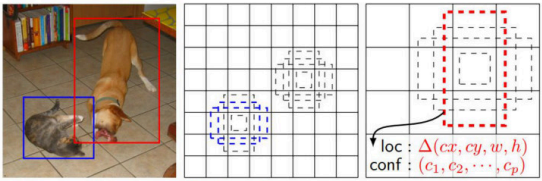
- There is **no** bounding-box proposal stage.
- Predictions are made at multiple scales, by predictors (filters) attached at different points in the network.

This results in

- A **significant speed improvement** over two-stage detectors, since there is no proposal stage: **59 FPS** on VOC2007.
- Accuracy on VOC2007 of **74.3% mAP**, compared with 63.4% mAP for YOLO at 45 FPS.

Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016.

The SSD framework



(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

Fig. 1: **SSD framework.** (a) SSD only needs an input image and ground truth boxes for each object during training. In a convolutional fashion, we evaluate a small set (e.g. 4) of default boxes of different aspect ratios at each location in several feature maps with different scales (e.g. 8×8 and 4×4 in (b) and (c)). For each default box, we predict both the shape offsets and the confidences for all object categories ($(c_1, c_2, \cdots, c_p)$). At training time, we first match these default boxes to the ground truth boxes. For example, we have matched two default boxes with the cat and one with the dog, which are treated as positives and the rest as negatives. The model loss is a weighted sum between localization loss (e.g. Smooth L1 [6]) and confidence loss (e.g. Softmax).

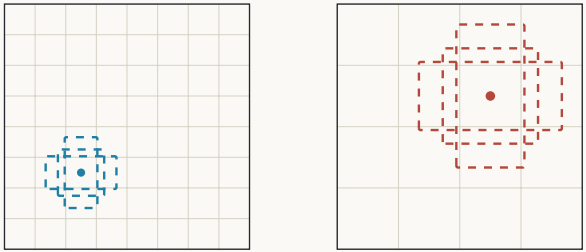
Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016, Fig. 1.

Default boxes at several scales

SSD uses a set of default boxes over different **scales**:

- Different feature maps correspond to different scales.
- Predictions from feature maps of several resolutions are combined.
- Fine maps hold small default boxes for small objects; coarse maps hold large ones.

default boxes of several aspect ratios, centered on one location of each map



8×8 feature map

4×4 feature map

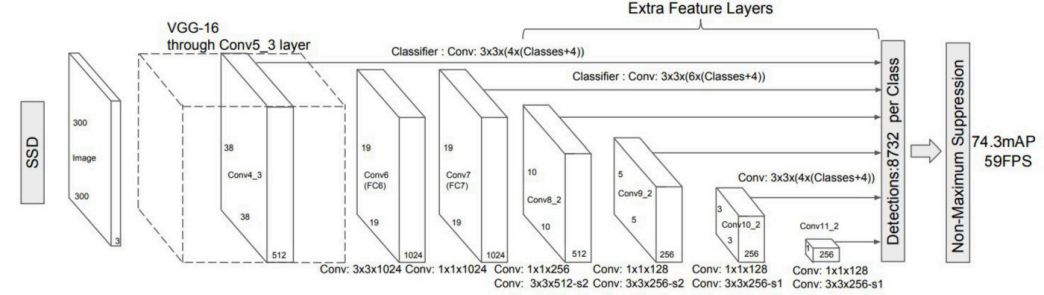
The core of SSD

Core idea

SSD predicts category scores and box offsets for a fixed set of default bounding boxes, using small convolutional filters applied to feature maps.

- For high detection accuracy, it makes predictions of different scales from feature maps of different scales, and explicitly separates predictions by aspect ratio.
- SSD reaches high accuracy even on low-resolution input images, which reduces the computational cost and increases the frame rate.

SSD architecture and multi-scale feature maps

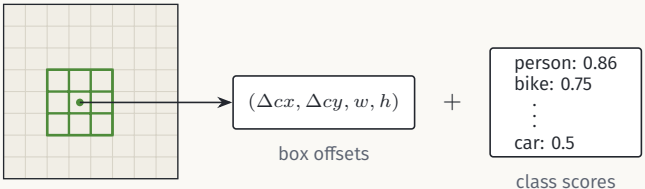


Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016, Fig. 2.

- Extra convolutional feature layers follow a truncated base network (VGG-16 through conv5_3).
- They shrink the feature map step by step, so detections are predicted at multiple scales.

Convolutional predictors for detection

- At each feature layer, a convolutional filter produces the predictions.
- For an $m \times n \times p$ feature map with k default boxes per location and c classes, the filter is $3 \times 3 \times p \times \big(k(c+4)\big)$.
- The output for an $m \times n$ feature map therefore has $(c+4)kmn$ values.



Example with $k = 4$ and $c = 21$: $(38 \times 38 \times 512) \xrightarrow{4\times3\times3\times512\times(21+4)} (38 \times 38 \times 4 \times (21+4))$

Matching default boxes to the ground truth

Matching strategy

For each ground-truth box, SSD selects default boxes that vary over location, aspect ratio, and scale.

1. Match each ground-truth box to the default box with the best Jaccard overlap (IoU).
2. Then match default boxes to any ground truth whose Jaccard overlap (IoU) is higher than a threshold of 0.5.

The SSD loss: localization

$$L(x, c, l, g) = \frac{1}{N}\Big(L_{\text{conf}}(x, c) + \alpha\, L_{\text{loc}}(x, l, g)\Big)$$

$$L_{\text{loc}}(x, l, g) = \sum_{i \in \text{Pos}}^{N} \sum_{m \in \{cx, cy, w, h\}} x_{ij}^{k}\ \text{smooth}_{L_1}\big(l_i^m - \hat{g}_j^m\big)$$

$$\hat{g}_j^{cx} = (g_j^{cx} - d_i^{cx})/d_i^w, \qquad \hat{g}_j^{cy} = (g_j^{cy} - d_i^{cy})/d_i^h$$

$$\hat{g}_j^{w} = \log\big(g_j^w / d_i^w\big), \qquad \hat{g}_j^{h} = \log\big(g_j^h / d_i^h\big)$$

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Symbols

$x_{ij}^{k} \in \{0, 1\}$ indicates that default box i is matched to ground-truth box j of category k.

N is the number of matched default boxes, l the predicted box, g the ground-truth box, and d the default box, each with center (cx, cy), width w, and height h.

Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016.

The SSD loss: confidence

The confidence loss is the softmax loss over the confidences c of the classes:

$$L_{\text{conf}}(x, c) = -\sum_{i \in \text{Pos}}^{N} x_{ij}^{p} \log\left(\hat{c}_i^p\right) - \sum_{i \in \text{Neg}} \log\left(\hat{c}_i^0\right) \qquad \text{where} \qquad \hat{c}_i^p = \frac{\exp(c_i^p)}{\sum_p \exp(c_i^p)}$$

- Matched (positive) boxes are pushed toward their class p; unmatched (negative) boxes toward the background class 0.
- The weight term α is set to 1 by cross-validation.

Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016.

Choosing scales and aspect ratios of default boxes

The scale of the default boxes on feature map k of m grows linearly:

$$s_k = s_{\min} + \frac{s_{\max} - s_{\min}}{m - 1}(k - 1), \qquad k \in [1, m]$$

In the paper, $s_{\min} = 0.2$ and $s_{\max} = 0.9$.

Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016.

For each aspect ratio a, the box has

$$w = s_k\sqrt{a}, \qquad h = \frac{s_k}{\sqrt{a}}$$

For aspect ratio 1, SSD adds one extra default box with scale

$$s'_k = \sqrt{s_k\, s_{k+1}}$$

the geometric mean of this scale and the next level's scale.

Hard negative mining

- After matching, most default boxes are negatives, especially when the number of possible default boxes is large.
- This creates a significant imbalance between positive and negative training examples.
- Instead of using all negatives, SSD sorts them by their confidence loss and keeps the highest ones, so that negatives outnumber positives at most 3 to 1.
- This leads to faster optimization and more stable training.

Datasets for comparison

Name	Type	Categories	Image size	Images per category	Objects per image
PASCAL VOC (2007, 2012)	Common objects	20	470×380	303–4087	2.4
MS COCO	Common objects	91	640×480	–	7.3
ImageNet (ILSVRC)	Common objects	200	500×400	–	1.5
KITTI	Driving scenes	8	1242×375	–*	–*
Cityscapes	Driving scenes	8	2048×1024	–*	–*

*Driving-scene datasets consist of continuous video frames and do not report images per category or objects per image.

SSD against YOLO and Faster R-CNN

Results on the VOC2007 test set

Method	mAP	FPS	Batch size	Boxes	Input resolution
Faster R-CNN (VGG16)	73.2	7	1	∼6000	$\sim 1000 \times 600$
Fast YOLO	52.7	155	1	98	448×448
YOLO (VGG16)	66.4	21	1	98	448×448
SSD300	74.3	46	1	8732	300×300
SSD512	**76.8**	19	1	24564	512×512
SSD300	74.3	**59**	8	8732	300×300
SSD512	**76.8**	22	8	24564	512×512

Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016.

Accuracy and speed on VOC2007

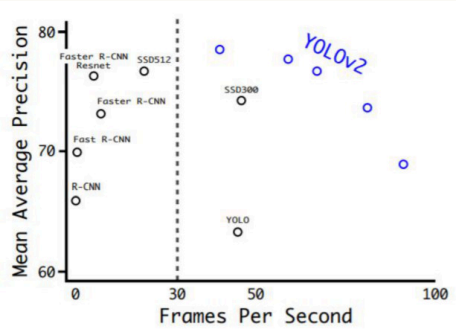


Figure 4: Accuracy and speed on VOC 2007.

Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017, Fig. 4.

- The dashed line at 30 FPS marks real-time speed.
- SSD300 and SSD512 are more accurate than YOLO, and SSD300 runs in real time.
- YOLOv2, covered next, improves on both.

PART 3

YOLO: You Only Look Once

Outline

- Fundamentals of object detection
- Evaluation criteria for object detection
- Conventional object detection methods
  - HOG DPM OverFeat
- Segmentation
  - Semantic and instance segmentation
- Evaluation criteria for semantic segmentation
- Early segmentation approaches
- Fully convolutional networks (FCNs)
- U-Net segmentation
- Region-based CNNs (R-CNN)
- Fast R-CNN
- Faster R-CNN
- Multi-task network cascades
- Mask R-CNN instance segmentation
- Single Shot MultiBox Detector (SSD), a single-stage detector
- **YOLO (You Only Look Once), a single-stage detector**
- RetinaNet

YOLO is a single-stage detector

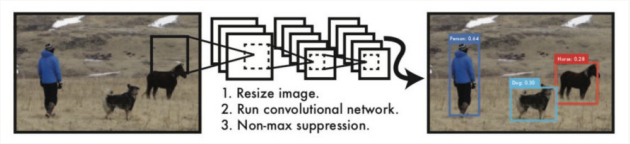
- A single neural network predicts bounding boxes and class probabilities from the full image in one evaluation.
- It is very fast, reaching up to 45 frames per second.
- It makes more localization errors than two-stage detectors, but is less likely to report false positives on the background.
- Because it sees the entire image, it implicitly encodes contextual information.

One look

Region-based detectors such as R-CNN classify many proposed regions one at a time. YOLO frames detection as a single regression problem, from image pixels straight to box coordinates and class probabilities.

Unified, real-time object detection

- A single convolutional network simultaneously predicts multiple bounding boxes and the class probabilities for those boxes.
- YOLO trains on full images and directly optimizes detection performance.
- Three steps: resize the image, run the network, apply non-max suppression.

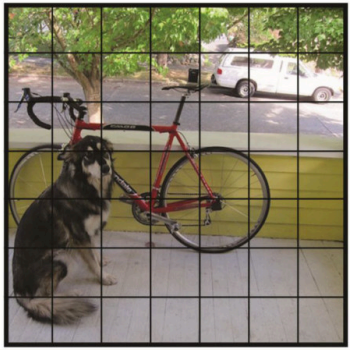


The YOLO detection system (1) resizes the input image to 448×448, (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

The image is divided into an $S \times S$ grid

- YOLO divides the input image into an $S \times S$ grid of cells.
- For evaluation on PASCAL VOC, which has 20 classes, the grid has 7×7 cells.
- If the center of an object falls into a grid cell, that cell is responsible for detecting the object.

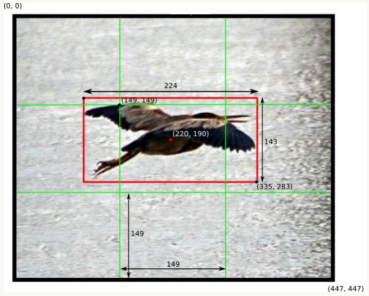


$S \times S$ grid on the input

Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

Each grid cell predicts B boxes with five numbers each

- Each cell predicts B bounding boxes ($B = 2$ in the paper).
- Each box has five components: x, y, w, h, and a confidence c.
- The confidence is $\Pr(\text{Object}) \cdot \text{IoU}^{\text{truth}}_{\text{pred}}$; its target is 1 for an object and 0 for no object.
- (x, y) is the box center relative to the bounds of the cell; w and h are relative to the whole image.

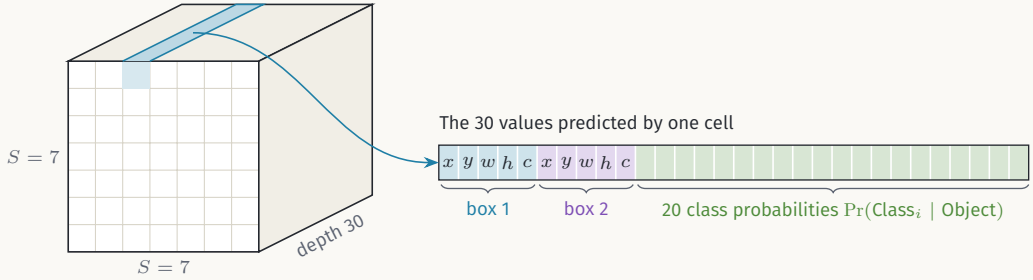


$$x = (220 - 149)/149 = 0.48 \qquad y = (190 - 149)/149 = 0.28$$
$$w = 224/448 = 0.50 \qquad h = 143/448 = 0.32$$

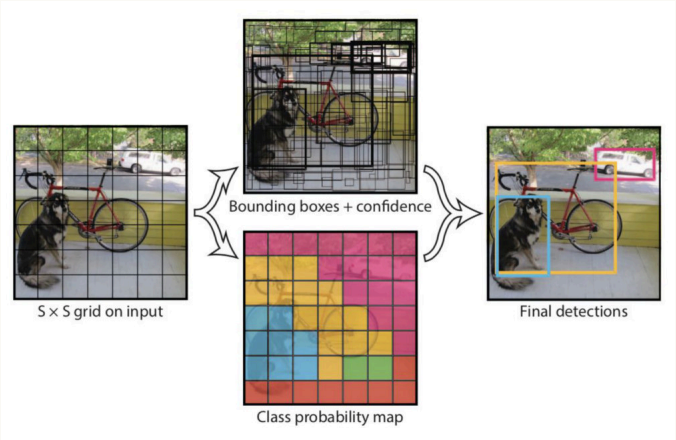
Grid cells in green, bounding box in red, on a 448×448 image divided into 3×3 cells. After hackernoon.com/understanding-yolo-f5a74bbc7967.

The network outputs a $7 \times 7 \times 30$ tensor

- With a 7×7 grid, each cell predicts 2 bounding boxes of 5 numbers each, plus 20 class probabilities.
- The network's output therefore has $7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30 = 1470$ values before non-max suppression.



YOLO processing steps



Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

From grid cells to class-specific scores

- Divide the input image into an $S \times S$ grid. Each cell predicts B bounding boxes and a confidence score for each box:

$$\text{confidence} = \Pr(\text{Object}) \cdot \text{IoU}_{\text{pred}}^{\text{truth}}.$$

- **Each bounding box** consists of 5 predictions: x, y, w, h, and the confidence.
- **Each cell** also predicts C conditional class probabilities, $\Pr(\text{Class}_i \mid \text{Object})$.
- Multiplying the conditional class probabilities by the box confidence gives a class-specific score for each box:

$$\Pr(\text{Class}_i \mid \text{Object}) \cdot \Pr(\text{Object}) \cdot \text{IoU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) \cdot \text{IoU}_{\text{pred}}^{\text{truth}}.$$

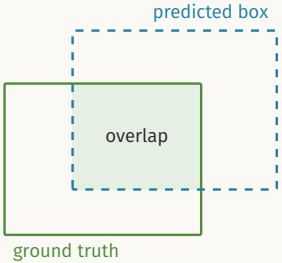
- All predictions are encoded in a tensor of size $S \times S \times (5B + C)$.

The confidence score combines objectness and overlap

In YOLO, the confidence score is formally defined as

$$\Pr(\text{Object}) \cdot \text{IoU}_{\text{pred}}^{\text{truth}}.$$

- **If no object exists in the cell**, the score should be 0.
- **Otherwise**, it should equal the IoU between the predicted box and the ground truth.



$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}}$$

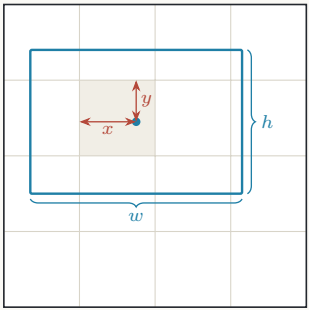
What the YOLO network predicts

- Each bounding box has 5 predictions: x, y, w, h, and confidence.
  - (x, y) is the center of the box relative to the bounds of the grid cell, $0 < x, y < 1$.
  - w and h are predicted relative to the entire image, $0 < w, h < 1$.
  - The confidence represents the IoU between the predicted box and the ground-truth box.
- Each grid cell also predicts C conditional class probabilities, $\Pr(\text{Class}_i \mid \text{Object})$.

Important

Only one set of class probabilities per grid cell, regardless of the number of boxes B.

x, y: offsets within the shaded cell



w, h: fractions of the image size

YOLO network design

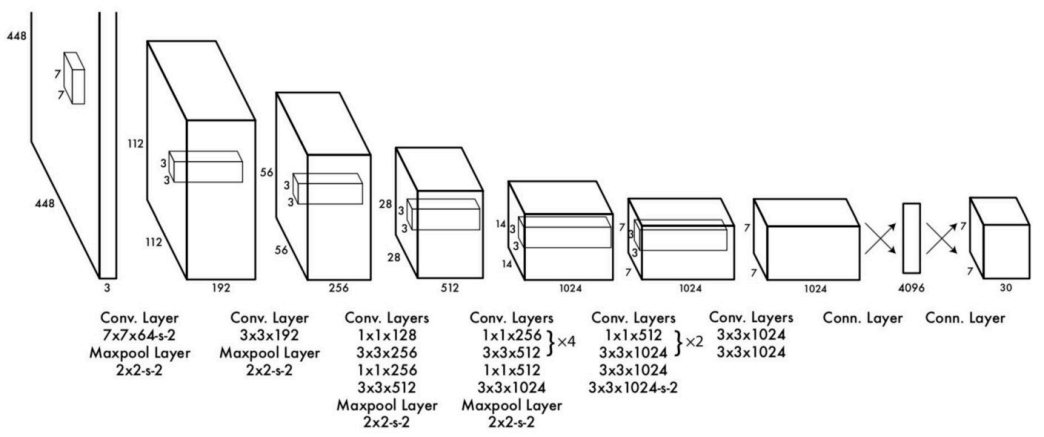
- The architecture is inspired by GoogLeNet.
- Detection requires fine-grained information, so the input resolution is increased from 224×224 to 448×448 pixels.
- The network has 24 convolutional layers followed by 2 fully connected layers.
- Instead of inception modules, it uses 1×1 reduction layers followed by 3×3 convolutional layers.

Output size

For $S = 7$, $B = 2$, and $C = 20$, the output is

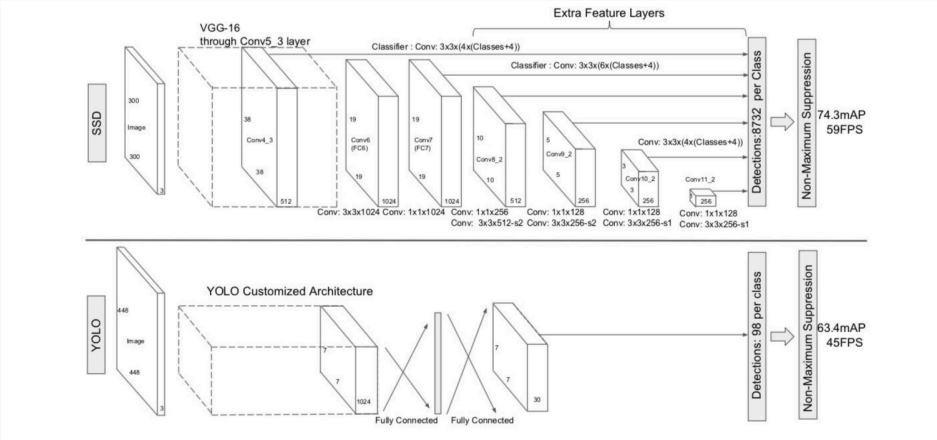
$$7 \times 7 \times 30, \qquad 30 = 5 \times 2 + 20.$$

YOLO network architecture



Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

SSD and YOLO networks compared



Liu et al., *SSD: Single Shot MultiBox Detector*, ECCV 2016.

Training YOLO: pre-train, then detect

- **First, pre-train** the convolutional layers on the ImageNet 1000-class competition dataset:
  - input size 224×224;
  - the first 20 convolutional layers of the network, followed by an average pooling layer and a fully connected layer.
- **Next, train for detection**: increase the input to 448×448 and add 4 convolutional layers and 2 fully connected layers with randomly initialized weights.
- The final layer predicts both class probabilities and bounding box coordinates.

Activations

The final layer uses a linear activation. All other layers use the **leaky ReLU**:

$$\phi(x) = \begin{cases} x, & \text{if } x > 0, \\ 0.1x, & \text{otherwise.} \end{cases}$$

The complete YOLO loss function

One predictor is assigned to be **responsible** for each object: the one whose prediction has the highest current IoU with the ground truth.

$\mathbf{1}_{ij}^{\text{obj}}$ is 1 when box predictor j of cell i is responsible for an object, and $\mathbf{1}_{ij}^{\text{noobj}}$ is 1 when it is not.

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

coordinate predictions

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i}\right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i}\right)^2 \right]$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i\right)^2$$

box confidence prediction

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i\right)^2$$

$$+ \sum_{i=0}^{S^2} \mathbf{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c)\right)^2$$

class prediction

Sum-squared error is easy to optimize, but imperfect

- **Design goal:** optimize the sum-squared error of the output.
  - It is **easy to optimize**.
  - However, it does not perfectly align with the real goal of maximizing average precision.
- It weighs localization error equally with classification error.
- Many grid cells do not contain any object. This pushes their confidence scores toward zero, often overpowering the gradient from the cells that do contain objects.

The consequence

Together, these factors can make training unstable.

Two weights and a square root keep training stable

Increase

the loss from bounding box coordinate predictions:

$$\lambda_{\text{coord}} = 5$$

Decrease

the loss from confidence predictions for boxes that do *not* contain objects:

$$\lambda_{\text{noobj}} = 0.5$$

- Sum-squared error also **weighs errors equally** in large and small bounding boxes.
- To partially address this, the loss compares the square roots of the box width and height instead of the width and height themselves.

Loss components: box center

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

- $\mathbf{1}_{i}^{\text{obj}}$ denotes whether an object appears in cell i.
- $\mathbf{1}_{ij}^{\text{obj}}$ denotes that the jth bounding box predictor in cell i is responsible for that prediction.
- (x_i, y_i) is the predicted center and $(\hat{x}_i, \hat{y}_i)$ the target, both relative to the grid cell.

Key idea

If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object.

Loss components: one responsible predictor per object

- YOLO predicts **multiple bounding boxes per grid cell**.
- At training time, only one bounding box predictor should be responsible for each object.
- The responsible predictor is the one whose prediction has the highest current IoU with the ground truth.

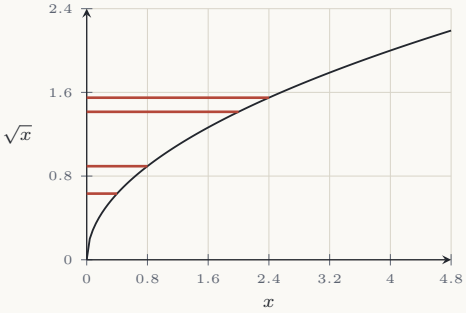
Specialization of the bounding box predictors

This assignment leads to specialization between the predictors. Each one gets better at predicting certain sizes, aspect ratios, or classes of objects, which improves overall recall.

Loss components: box width and height

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

- **Sum-squared error** weighs errors in large and small boxes equally.
- The metric should reflect that a small deviation matters less in a large box than in a small one.
- To partially address this, YOLO predicts the square root of the box width and height instead of the width and height directly.



The same change in size, 0.4, moves $\sqrt{x}$ much more for a small box (0.4 to 0.8) than for a large box (2.0 to 2.4).

Loss components: confidence and class probabilities

Confidence

$$\sum_{i=0}^{S^2}\sum_{j=0}^{B}\mathbf{1}_{ij}^{\text{obj}}\left(C_i-\hat{C}_i\right)^2$$
$$+\lambda_{\text{noobj}}\sum_{i=0}^{S^2}\sum_{j=0}^{B}\mathbf{1}_{ij}^{\text{noobj}}\left(C_i-\hat{C}_i\right)^2$$

The first term covers responsible predictors; the second covers predictors with no object, down-weighted by λ_{noobj}.

$\Pr(\textbf{Class}_i \mid \textbf{Object})$

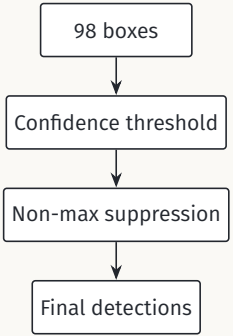
$$\sum_{i=0}^{S^2}\mathbf{1}_{i}^{\text{obj}}\sum_{c\in\text{classes}}\left(p_i(c)-\hat{p}_i(c)\right)^2$$

The classification error is counted only for cells that contain an object, once per cell rather than once per box.

YOLO inference

General procedure

1. Predict 98 bounding boxes per image (7×7 cells, 2 boxes each).
2. Discard bounding boxes with low confidence.
3. Use non-max suppression to combine the remaining predictions.



Advantages of YOLO

Extremely fast

- 45 FPS on a Titan X GPU.
- 155 FPS for Fast YOLO, which has 9 convolutional layers instead of 24 and fewer filters per layer.

Reasons globally

YOLO sees the entire image during training and test time, so it encodes context about classes and their appearance.

Generalizes

YOLO learns generalizable representations of objects, so it transfers better to new domains such as artwork.

Drawbacks of YOLO

Strong spatial constraints on bounding boxes

- Each grid cell predicts only B boxes (by default $B = 2$).
- Each grid cell can have only one class.
- Several objects can share a cell, such as a person standing in front of a car.

Trouble with new aspect ratios or configurations

- Bounding boxes have weak priors (no anchors).
- Features are coarse because of the many downsampling layers.

Important

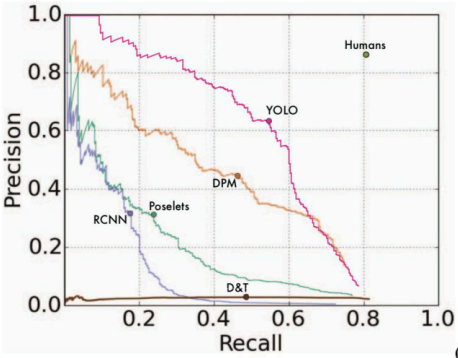
The model struggles with small objects that appear in groups, such as flocks of birds. Most of its errors are incorrect localizations, especially of small objects.

Real-time detectors on PASCAL VOC 2007

Real-time detectors	Train	mAP	FPS
100Hz DPM	2007	16.0	100
30Hz DPM	2007	26.1	30
Fast YOLO	2007+2012	52.7	**155**
YOLO	2007+2012	**63.4**	45
Less than real-time			
Fastest DPM	2007	30.4	15
R-CNN Minus R	2007	53.5	6
Fast R-CNN	2007+2012	70.0	0.5
Faster R-CNN VGG-16	2007+2012	73.2	7
Faster R-CNN ZF	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

YOLO generalizes to artwork



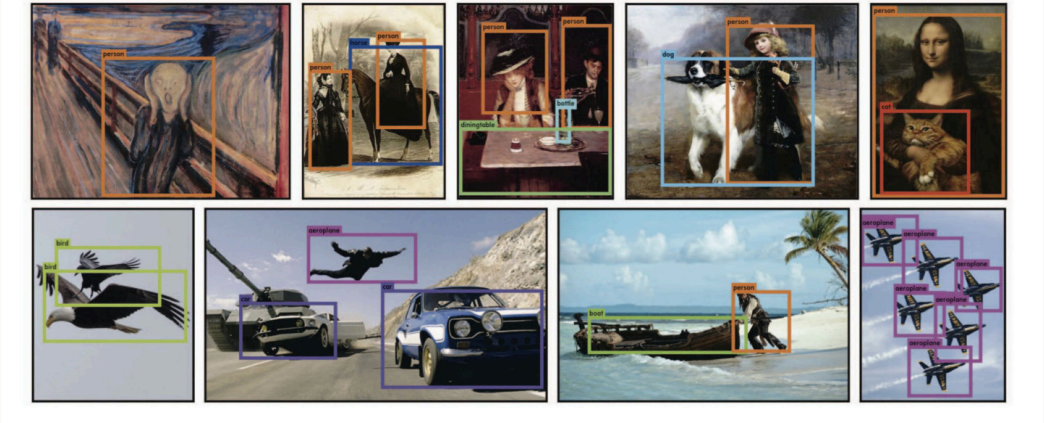
(a) Picasso Dataset precision-recall curves.

	VOC 2007	Picasso		People-Art
	AP	AP	Best F_1	AP
YOLO	**59.2**	**53.3**	**0.590**	**45**
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

YOLO examples on artwork and natural images



Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

Comparison on the PASCAL VOC 2012 leaderboard

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE_DATA [11]	**73.9**	**85.5**	**82.9**	**76.6**	**57.8**	**62.7**	**79.4**	77.2	86.6	**55.0**	**79.1**	**62.2**	87.0	**83.4**	**84.7**	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	**79.8**	87.7	49.6	74.9	52.1	86.0	81.7	83.3	**81.8**	**48.6**	**73.5**	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	**89.4**	49.4	75.5	57.0	**87.5**	80.9	81.0	74.7	41.8	71.5	68.5	**82.1**	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	**68.8**	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	**87.5**	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

YOLO compared with the full `comp4` (outside data allowed) public leaderboard as of November 6, 2015: mean and per-class average precision. YOLO is the only real-time detector. Fast R-CNN + YOLO is the fourth-highest scoring method, with a 2.3% boost over Fast R-CNN.

Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, CVPR 2016.

PART 4

YOLO9000 and YOLOv2

YOLO9000 sets out to fix YOLOv1's errors

The problem

YOLOv1 has high localization error (wrong boxes) and low recall (it misses objects in the image).

The goals

Improve recall, localization, and classification accuracy.

First improvements

- **Batch normalization** on all convolutional layers increases mAP by about 2%. It also regularizes the model, so dropout is removed.
- **High-resolution classifier**: fine-tune the classification network at the full 448×448 resolution for 10 epochs.
- **Convolution with anchor boxes**:
  - remove the fully connected layers and predict boxes with anchor boxes;
  - remove one pooling layer for a higher-resolution output;
  - shrink the input from 448×448 to 416×416.

Improving YOLOv1: fine-grained features and multi-scale training

Fine-grained features

- A passthrough layer brings features from an earlier layer at 26×26 resolution.
- These are concatenated with the low-resolution 13×13 features by stacking adjacent features into different channels: $26 \times 26 \times 512$ becomes $13 \times 13 \times 2048$.

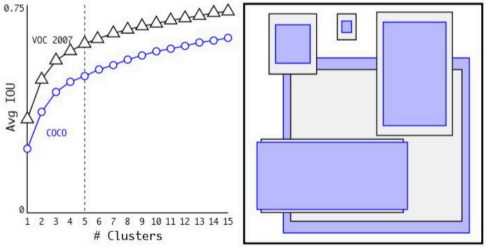
Multi-scale training

- The model uses only convolutional and pooling layers, so it can be resized on the fly.
- Every 10 batches, the input image size changes.
- The model downsamples by 32, so sizes are drawn from $\{320, 352, \ldots, 608\}$.

Improving YOLOv1: anchor box dimensions from clusters

- The dimensions of anchor boxes are usually hand-picked.
- Instead, run k-means clustering on the bounding boxes of the training set to find good priors automatically.

Left: average IoU for different choices of k; $k = 5$ gives a good tradeoff between recall and model complexity. Right: relative centroids for VOC and COCO. Both sets of priors favor thinner, taller boxes, and COCO has greater variation in size than VOC.



Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017.

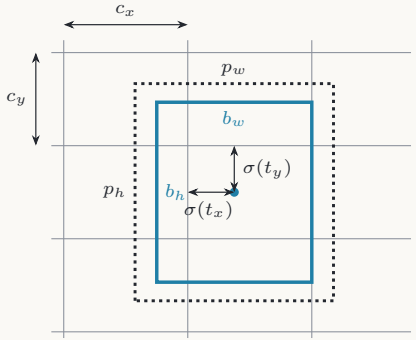
Direct location prediction

- For each anchor box the network predicts t_x, t_y, t_w, t_h, t_o; this parameterization keeps training stable.
- The cell is offset by (c_x, c_y) from the image's top left; the prior has size $p_w \times p_h$.

$$b_x = \sigma(t_x) + c_x \qquad b_w = p_w e^{t_w}$$
$$b_y = \sigma(t_y) + c_y \qquad b_h = p_h e^{t_h}$$

$$\Pr(\text{object}) \cdot \text{IoU}(b, \text{object}) = \sigma(t_o)$$

Dimension clusters with direct location prediction improve YOLO by almost 5% over the version with anchor boxes.

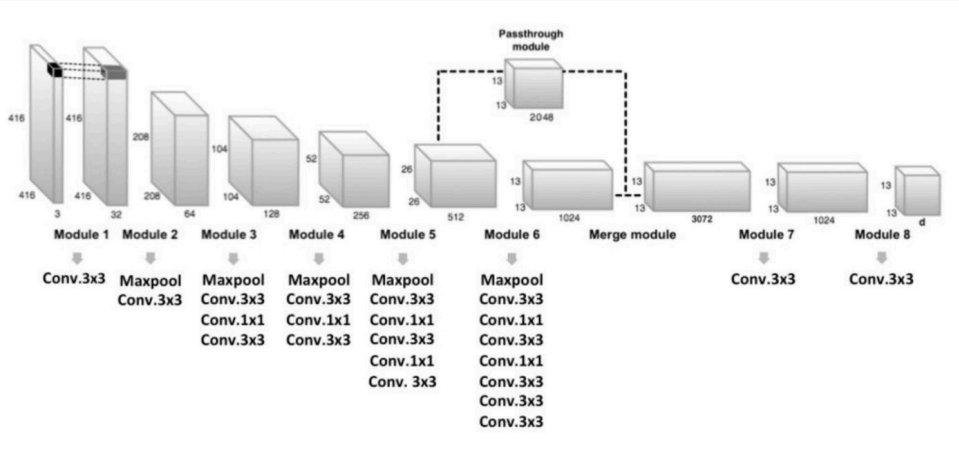


Dotted: the anchor prior; blue: the predicted box. After Redmon and Farhadi, CVPR 2017, Figure 3; see also the datlife/yolov2 implementation.

How YOLOv2 improves on YOLOv1

- Remove the last fully connected layer and use a convolutional layer for prediction.
- Resize the input to 416×416. Since $416/32 = 13$, the output grid is 13×13, an odd number of cells with a single center cell. Objects, especially large ones, tend to sit at the center of the image.
- Predict $\Pr(\text{class} \mid \text{object})$ and a confidence for every anchor box.

YOLOv2 architecture



The path from YOLOv1 to YOLOv2

	YOLO								YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓	✓
anchor boxes?				✓	✓				
new network?					✓	✓	✓	✓	✓
dimension priors?						✓	✓	✓	✓
location prediction?						✓	✓	✓	✓
passthrough?							✓	✓	✓
multi-scale?								✓	✓
hi-res detector?									✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8	**78.6**

Most design decisions lead to significant increases in mAP. Two exceptions are switching to a fully convolutional network with anchor boxes and using the new network. Switching to anchor boxes increased recall without changing mAP, while the new network cut computation by 33%.

Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017.

Darknet-19, the backbone of YOLOv2

19 convolutional layers and 5 max-pooling layers, fewer than YOLOv1, with 72.9% top-1 and 91.2% top-5 accuracy on ImageNet.

Type	Filters	Size/stride	Output
Convolutional	32	3×3	224×224
Maxpool		$2 \times 2/2$	112×112
Convolutional	64	3×3	112×112
Maxpool		$2 \times 2/2$	56×56
Convolutional	128	3×3	56×56
Convolutional	64	1×1	56×56
Convolutional	128	3×3	56×56
Maxpool		$2 \times 2/2$	28×28
Convolutional	256	3×3	28×28
Convolutional	128	1×1	28×28
Convolutional	256	3×3	28×28
Maxpool		$2 \times 2/2$	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Maxpool		$2 \times 2/2$	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017.

Training YOLOv2 in two steps

1. Training for classification

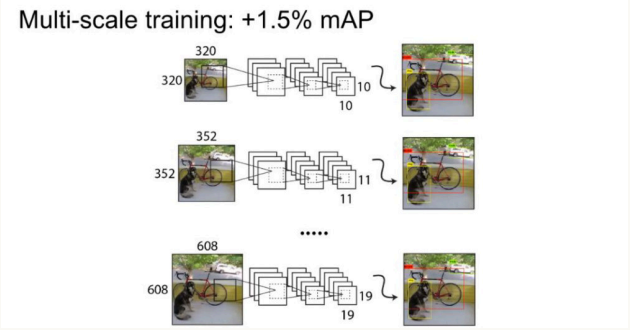
- Train on the ImageNet 1000-class dataset for 160 epochs with the Darknet framework.
- SGD with a starting learning rate of 0.1, polynomial rate decay with a power of 4, weight decay of 0.0005, and momentum of 0.9.
- Augmentation: random crops, rotations, and hue, saturation, and exposure shifts.
- After initial training at 224×224, fine-tune for 10 epochs at 448×448.

2. Training for detection

- Remove the last convolutional layer and add three 3×3 convolutional layers with 1024 filters each, followed by a final 1×1 convolutional layer.
- Add a passthrough layer, which does not exist in classification training.

YOLOv2 multi-scale training

- Train with a different input size every 10 batches.
- Sizes range from 320 to 608, in multiples of 32.
- Multi-scale training improves mAP by 1.5%.
- One network then trades speed for accuracy at test time, simply by changing the input size.



Multi-scale training in YOLOv2 (Redmon and Farhadi, CVPR 2017).

YOLOv2 results on PASCAL VOC 2012

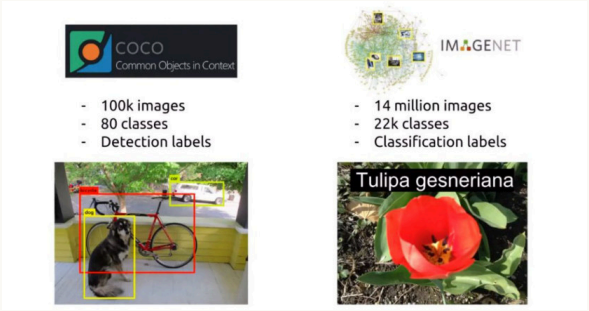
Method	data	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
Fast R-CNN [5]	07++12	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
Faster R-CNN [15]	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
YOLO [14]	07++12	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
SSD300 [11]	07++12	72.4	85.6	80.1	70.5	57.6	46.2	79.4	76.1	89.2	53.0	77.0	60.8	87.0	83.1	82.3	79.4	45.9	75.9	69.5	81.9	67.5
SSD512 [11]	07++12	74.9	87.4	82.3	75.8	59.0	52.6	81.7	81.5	90.0	55.4	79.0	59.8	88.4	84.3	84.7	83.3	50.2	78.0	66.3	86.3	72.0
ResNet [6]	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
YOLOv2 544	07++12	73.4	86.3	82.0	74.8	59.2	51.8	79.8	76.5	90.6	52.1	78.2	58.5	89.3	82.5	83.4	81.3	49.1	77.2	62.4	83.8	68.7

PASCAL VOC 2012 `test` detection results. YOLOv2 performs on par with state-of-the-art detectors like Faster R-CNN with ResNet and SSD512, and is 2 to 10 times faster.

Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017.

From YOLOv2 to YOLO9000

- YOLO9000 is a version of YOLOv2 that detects a very large number of classes: more than 9000 object categories.
- In comparison, YOLO trained on COCO knows only **80** classes.



YOLO9000: joint training for detection and classification

Detection images

Images labeled for detection teach detection-specific information, such as bounding box coordinates and objectness.

Classification images

Images with only class labels expand the number of categories the model can detect.

The challenge: labels at different levels

- Detection datasets have only a few object types, with generic labels.
- Classification datasets have a much wider and deeper range of labels.



Detection label: **dog**

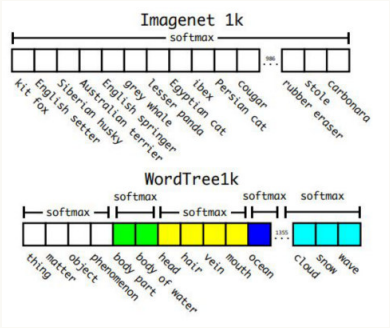


Classification label: **Norfolk terrier**

Jointly training classification and detection

- For detection images, backpropagate the full loss; for classification images, only the classification loss.
- A detection “dog” is a classification “Norfolk terrier”, “Yorkshire terrier”, or “Bedlington terrier”. The labels are not mutually exclusive, so one softmax cannot be used directly.
- **Approach:** build a multi-label model, giving one object several labels.

$$\Pr(\text{Norfolk terrier}) = \Pr(\text{Norfolk terrier} \mid \text{terrier}) \cdot \Pr(\text{terrier} \mid \text{hunting dog}) \cdot \ldots \cdot \Pr(\text{mammal} \mid \text{animal}) \cdot \Pr(\text{animal} \mid \text{physical object})$$

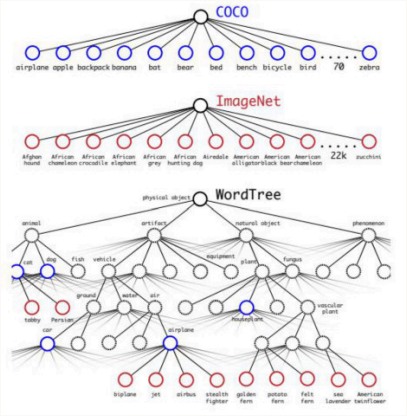


ImageNet uses one softmax over all classes; WordTree uses one softmax per group of co-hyponyms. Redmon and Farhadi, CVPR 2017.

Combining datasets with the WordTree hierarchy

- The WordNet concept graph is used to build a hierarchical tree of visual concepts.
- Datasets are then merged by mapping their classes to synsets in the tree.

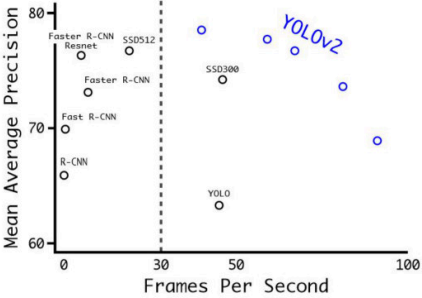
The figure is a simplified view of WordTree for illustration.



Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017.

YOLOv2 results: speed and accuracy

Detection framework	Train	mAP	FPS
Fast R-CNN	2007+2012	70.0	0.5
Faster R-CNN VGG-16	2007+2012	73.2	7
Faster R-CNN ResNet	2007+2012	76.4	5
YOLO	2007+2012	63.4	45
SSD300	2007+2012	74.3	46
SSD500	2007+2012	76.8	19
YOLOv2 288×288	2007+2012	69.0	91
YOLOv2 352×352	2007+2012	73.7	81
YOLOv2 416×416	2007+2012	76.8	67
YOLOv2 480×480	2007+2012	77.8	59
YOLOv2 544×544	2007+2012	**78.6**	40



Detection frameworks on PASCAL VOC 2007: accuracy against frames per second.

Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017.

YOLOv2: some details

- Class predictions are no longer made per cell; they are made per bounding box.
- The network predicts 5 bounding boxes per cell of the output feature map.
- For each box it predicts 5 coordinates: t_x, t_y, t_w, t_h, and t_o.

Compared with YOLOv1

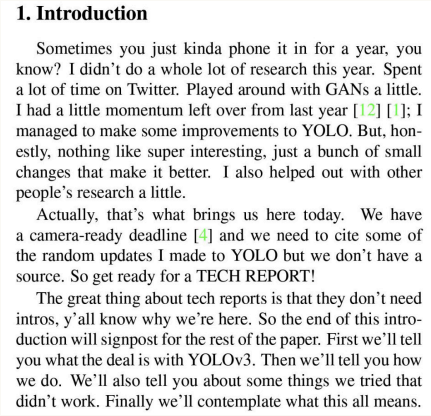
YOLOv1 shares one set of class probabilities among the $B = 2$ boxes of a cell. YOLOv2 gives each of its 5 anchor boxes its own class probabilities and objectness score.

PART 5

YOLOv3

YOLOv3: an incremental improvement

- Redmon and Farhadi presented YOLOv3 as a short tech report rather than a full paper.
- The introduction sets the tone: no single big idea, just a bunch of small design changes that make YOLO better.
- The report then covers what changed, how it performs, and what did not work.



1. Introduction

Sometimes you just kinda phone it in for a year, you know? I didn't do a whole lot of research this year. Spent a lot of time on Twitter. Played around with GANs a little. I had a little momentum left over from last year [12] [1]; I managed to make some improvements to YOLO. But, honestly, nothing like super interesting, just a bunch of small changes that make it better. I also helped out with other people's research a little.

Actually, that's what brings us here today. We have a camera-ready deadline [4] and we need to cite some of the random updates I made to YOLO but we don't have a source. So get ready for a TECH REPORT!

The great thing about tech reports is that they don't need intros, y'all know why we're here. So the end of this introduction will signpost for the rest of the paper. First we'll tell you what the deal is with YOLOv3. Then we'll tell you how we do. We'll also tell you about some things we tried that didn't work. Finally we'll contemplate what this all means.

Redmon and Farhadi, *YOLOv3: An Incremental Improvement*, arXiv 2018.

A bunch of little design changes make YOLO better

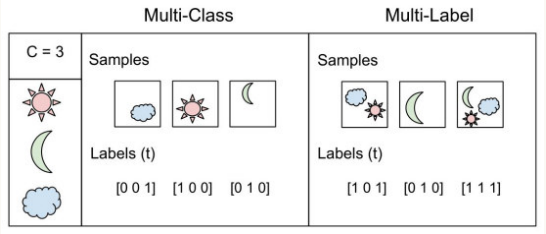
- At 320×320, YOLOv3 runs in 22 ms at 28.2 mAP: as accurate as SSD and three times faster.
- It reaches 57.9 AP_{50} in 51 ms on a Titan X, compared with 57.5 AP_{50} in 198 ms for RetinaNet: similar performance, $3.8\times$ faster.
- Code: pjreddie.com/darknet/yolo
- Video: youtube.com/watch?v=MPU2HistivI

Three significant changes

1. The backbone is more ResNet-like and is now called Darknet-53.
2. Each box predicts the classes it may contain with multi-label classification.
3. Predictions are made at three different scales.

Class prediction uses independent logistic classifiers

- Each box predicts the classes it may contain with **multi-label** classification.
- There is no softmax: the authors found it unnecessary for good performance and use independent logistic classifiers instead.
- Each class output passes through its own sigmoid, a yes-or-no question per class.
- Labels can overlap (*woman* and *person*), which a softmax would forbid.



Gómez, *Understanding categorical cross-entropy loss*, 2018.

$$\hat{p}_c = \sigma(z_c) = \frac{1}{1 + e^{-z_c}} \quad \text{for each class } c$$

Predictions across scales and a new feature extractor

Predictions across scales

YOLOv3 predicts boxes at three different scales, extracting features from those scales with a concept similar to feature pyramid networks.

Feature extractor

A hybrid of YOLOv2's Darknet-19 and residual networks: successive 3×3 and 1×1 convolutions with shortcut connections, and significantly larger. It has 53 convolutional layers, so it is called Darknet-53.

Paper: pjreddie.com/media/files/papers/YOLOv3.pdf Author's site: pjreddie.com

Three output resolutions, three anchors each

- The network produces feature maps at three resolutions.
- With a 416×416 input, the coarsest map is scaled down by 32.
- The maps are 13×13, 26×26, and 52×52; higher resolution serves smaller objects.
- Each resolution has 3 anchor boxes, so there are 9 anchors in total.

Output tensor

On COCO, each cell predicts 3 boxes, each with 4 box offsets, 1 objectness score, and 80 class scores:

$$N \times N \times \big[3 \times (4+1+80)\big] = N \times N \times 255.$$

Darknet-53 learns to classify before it detects

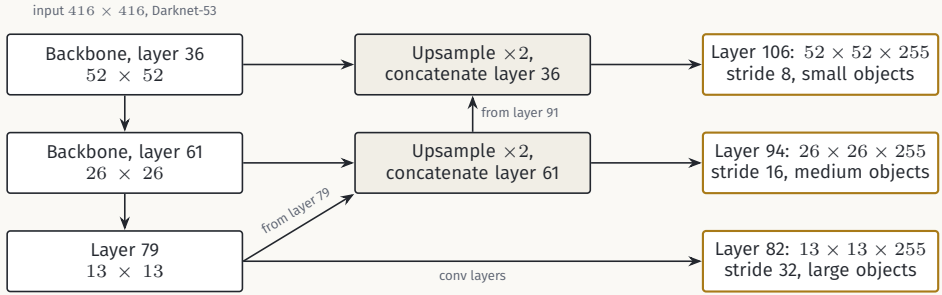
1. Train the network for classification at 256×256.
2. Change the input to 416×416 and fine-tune for classification.
3. Switch to detection on COCO: 80 classes, 3 boxes per cell, $N \times N$ cells, $N \times N \times [3 \times (4 + 1 + 80)]$ outputs per image.

At a 416×416 input, the stages with strides 8, 16, and 32 feed the three detection scales.

	Type	Filters	Size	Output	
	Conv	32	3×3	256×256	
	Conv	64	$3 \times 3/2$	128×128	
$1\times$	Conv, conv, residual	32, 64	$1 \times 1, 3 \times 3$	128×128	
	Conv	128	$3 \times 3/2$	64×64	
$2\times$	Conv, conv, residual	64, 128	$1 \times 1, 3 \times 3$	64×64	
	Conv	256	$3 \times 3/2$	32×32	
$8\times$	Conv, conv, residual	128, 256	$1 \times 1, 3 \times 3$	32×32	stride 8
	Conv	512	$3 \times 3/2$	16×16	
$8\times$	Conv, conv, residual	256, 512	$1 \times 1, 3 \times 3$	16×16	stride 16
	Conv	1024	$3 \times 3/2$	8×8	
$4\times$	Conv, conv, residual	512, 1024	$1 \times 1, 3 \times 3$	8×8	stride 32
	Avgpool, connected, softmax		Global	1000	

Darknet-53, after Redmon and Farhadi 2018, Table 1. Output sizes are for the 256×256 classification input.

Three detection layers, from coarse to fine



- The final output, layer 82, is scaled down by 32: a 416×416 input gives 13×13, used for large objects.
- Upsampling to 26×26 and concatenating layer 61 gives the medium scale; upsampling again to 52×52 and concatenating layer 36 gives the small scale.

Layer numbers follow github.com/pjreddie/darknet/blob/master/cfg/yolov3.cfg.

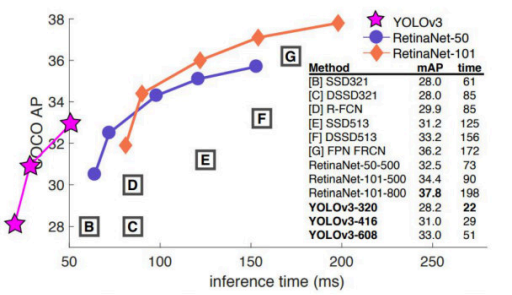
Darknet-53 rivals ResNet-152 at twice the speed

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19	74.1	91.8	7.29	1246	**171**
ResNet-101	77.1	93.7	19.7	1039	53
ResNet-152	**77.6**	**93.8**	29.4	1090	37
Darknet-53	77.2	**93.8**	18.7	**1457**	78

ImageNet classification, Redmon and Farhadi 2018, Table 2.

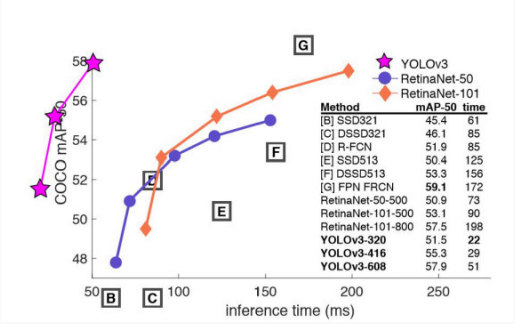
- Darknet-53 is as accurate as ResNet-152 on Top-5 and runs about twice as fast.
- It achieves the highest measured floating-point operations per second, so it uses the GPU efficiently.

YOLOv3 is far faster at comparable accuracy



Method	mAP	time
[B] SSD321	28.0	61
[C] DSSD321	28.0	85
[D] R-FCN	29.9	85
[E] SSD513	31.2	125
[F] DSSD513	33.2	156
[G] FPN FRCN	36.2	172
RetinaNet-50-500	32.5	73
RetinaNet-101-500	34.4	90
RetinaNet-101-800	**37.8**	198
YOLOv3-320	28.2	**22**
YOLOv3-416	31.0	29
YOLOv3-608	33.0	51

Figure 1. We adapt this figure from the Focal Loss paper [9]. YOLOv3 runs significantly faster than other detection methods with comparable performance. Times from either an M40 or Titan X, they are basically the same GPU.



Method	mAP-50	time
[B] SSD321	45.4	61
[C] DSSD321	46.1	85
[D] R-FCN	51.9	85
[E] SSD513	50.4	125
[F] DSSD513	53.3	156
[G] FPN FRCN	**59.1**	172
RetinaNet-50-500	50.9	73
RetinaNet-101-500	53.1	90
RetinaNet-101-800	57.5	198
YOLOv3-320	51.5	**22**
YOLOv3-416	55.3	29
YOLOv3-608	57.9	51

Left: COCO AP versus inference time, adapted by the authors from the focal loss paper. Right: the same comparison at mAP_{50}. Redmon and Farhadi 2018, Figures 1 and 3.

YOLOv3 trails at strict IoU thresholds

	Backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI	Inception-ResNet-v2	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2	DarkNet-19	21.6	44.0	19.2	5.0	22.4	35.5
SSD513	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2
YOLOv3 608×608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

COCO test-dev, Redmon and Farhadi 2018, Table 3.

What did not work in YOLOv3

Anchor box x, y offset predictions

Predicting the x, y offset as a multiple of the box width or height with a linear activation decreased model accuracy and stability. Using a linear activation by itself did not work either.

Dual IoU thresholds

Following Faster R-CNN, a prediction overlapping the ground truth by 0.7 or more is a positive example, by 0.3 to 0.7 it is ignored, and below 0.3 it is a negative example. This assignment did not work well for YOLOv3.

PART 6

YOLOv4

YOLOv4: a fast detector you can train on one GPU

- YOLOv4 brings together several ideas that improve performance, around one core goal: a CNN that
  1. runs in real time on a conventional GPU, and
  2. trains on a single conventional GPU,

  by optimizing for parallel computation rather than for the theoretical computation volume (BFLOP).

> **Gain at 416×416 on a Titan X**
>
> YOLOv3: 35 FPS at 55.3% AP_{50}. YOLOv4: 38 FPS at 62.8% AP_{50}.

Paper: arxiv.org/abs/2004.10934 Code: github.com/AlexeyAB/darknet

The anatomy of an object detector

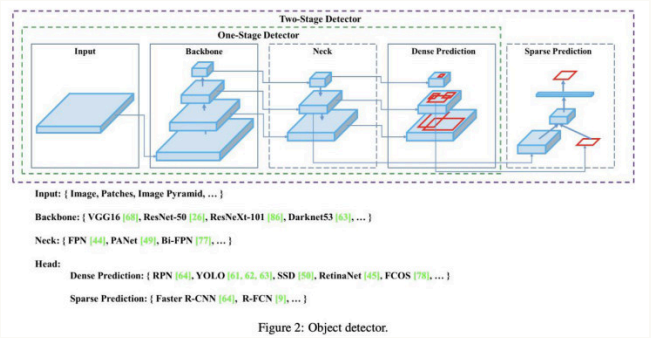


Figure 2: Object detector.

Bochkovskiy, Wang, and Liao, *YOLOv4: Optimal Speed and Accuracy of Object Detection*, arXiv 2020, Figure 2.

YOLOv4 consists of

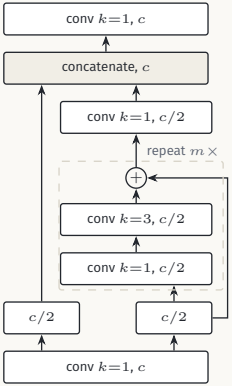
- **Backbone**: CSPDarknet53
- **Neck**: SPP and PAN
- **Head**: YOLOv3

The backbone: CSPDarknet53

- The **CSPNet** strategy partitions the feature map of the base layer into two parts and merges them through a cross-stage hierarchy.
- The split-and-merge strategy allows more gradient flow.
- CSP saves computation across many CNN architectures.

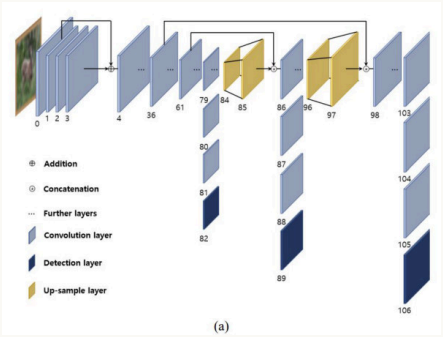
Backbone	Receptive field	Parameters	Avg. layer output	BFLOPs	FPS
CSPResNeXt50	425×425	20.6 M	**1058 K**	31 (15.5 FMA)	62
CSPDarknet53	725×725	**27.6 M**	950 K	52 (26.0 FMA)	**66**
EfficientNet-B3	$\mathbf{1311 \times 1311}$	12.0 M	668 K	11 (5.5 FMA)	26

Input 512×512, FPS on an RTX 2070. Bochkovskiy et al. 2020, Table 1.

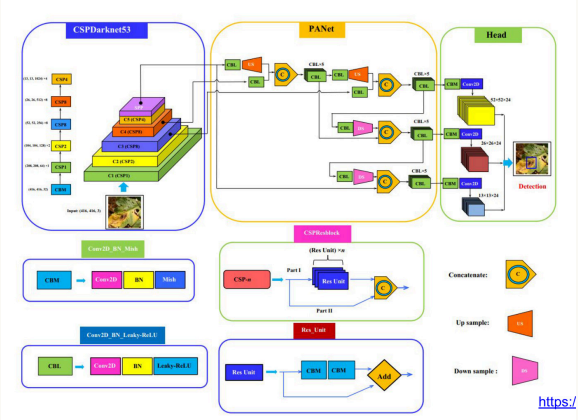


CSP block, after Bochkovskiy et al. 2020.

From the YOLOv3 architecture to YOLOv4



YOLOv3.



YOLOv4: CSPDarknet53 backbone, PANet neck, YOLO head.

Both from arxiv.org/abs/2111.00298.

A CSP block in code

```
def dense_block_cspnet(inps, partition, filter, times, id):
    shape = inps.shape
    features = shape[3] - partition
    part1 = inps[:, :, :, 0:features]
    part2 = inps[:, :, :, features:]
    for time in range(0, times):
        part2 = Conv2D(filter[0], 1, padding="same")(part2)
        part2 = mish(part2)
        part2 = Conv2D(filter[1], 3, padding="same")(part2)
        part2 = mish(part2)

    part2 = Conv2D(filter[2], 1, padding="same")(part2)
    part2 = mish(part2)
    inps = Concatenate()([part1, part2])
    return inps
```

- The channels split into `part1` and `part2`.
- Only `part2` goes through the repeated 1×1 and 3×3 convolutions.
- `part1` skips the block and rejoins at the concatenation.

Spatial pyramid pooling gives a fixed-size output

Problem

A CNN's output size depends on its input size.

Solution

Spatial pyramid pooling (SPP) creates a fixed-size feature vector from an arbitrarily sized input image.

This helps in both classification and detection.

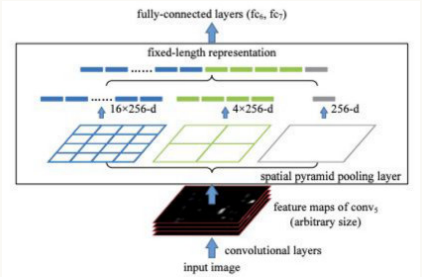


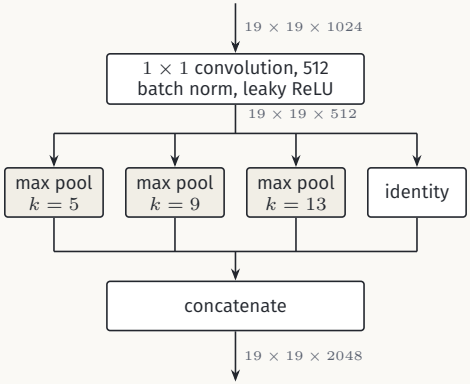
Figure 3: A network structure with a **spatial pyramid pooling layer**. Here 256 is the filter number of the $conv_5$ layer, and $conv_5$ is the last convolutional layer.

He, Zhang, Ren, and Sun, *Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition*, ECCV 2014, Figure 3.

SPP in YOLOv4 enlarges the receptive field

- Max-pooling runs over the $19 \times 19 \times 512$ feature map, alongside a branch with no pooling.
- The kernel sizes are $k \in \{5, 9, 13\}$, with stride 1 and *same* padding so the spatial size stays 19×19.
- Concatenating the four branches gives $19 \times 19 \times 2048$.

After *YOLO v4: Optimal speed and accuracy for object detection*, Towards Data Science.



PANet aggregates features across backbone levels

Path Aggregation Network

A method for aggregating parameters from different backbone levels for different detector levels.

YOLOv3 used FPN instead of PAN.

FPN: arxiv.org/abs/1612.03144

PAN: Liu et al., CVPR 2018

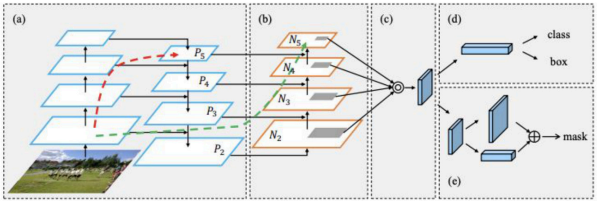


Figure 1. Illustration of our framework. (a) FPN backbone. (b) Bottom-up path augmentation. (c) Adaptive feature pooling. (d) Box branch. (e) Fully-connected fusion. Note that we omit channel dimension of feature maps in (a) and (b) for brevity.

(a) FPN backbone, (b) bottom-up path augmentation, (c) adaptive feature pooling, (d) box branch, (e) fully connected fusion. Liu, Qi, Qin, Shi, and Jia, *Path Aggregation Network for Instance Segmentation*, CVPR 2018, Figure 1.

FPN, PANet, and later feature network designs

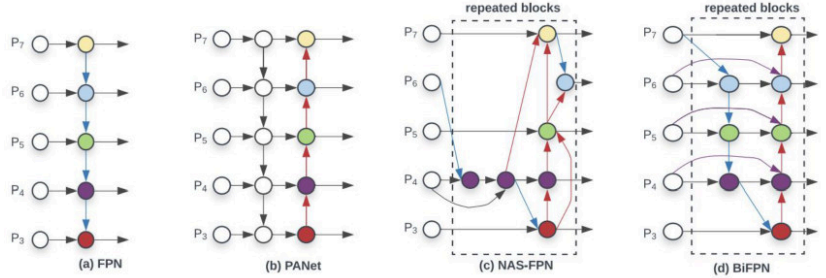


Figure 2: **Feature network design –** (a) FPN [23] introduces a top-down pathway to fuse multi-scale features from level 3 to 7 (P_3 - P_7); (b) PANet [26] adds an additional bottom-up pathway on top of FPN; (c) NAS-FPN [10] use neural architecture search to find an irregular feature network topology and then repeatedly apply the same block; (d) is our BiFPN with better accuracy and efficiency trade-offs.

(a) FPN adds a top-down pathway; (b) PANet adds a bottom-up pathway on top of FPN; (c) NAS-FPN; (d) BiFPN. Tan, Pang, and Le, *EfficientDet: Scalable and Efficient Object Detection*, CVPR 2020, Figure 2, arxiv.org/abs/1911.09070.

The head is still anchor-based, as in YOLOv3

The network predicts offsets t_x, t_y, t_w, t_h relative to an anchor (prior) of width p_w and height p_h in the cell with top-left corner (c_x, c_y):

$$b_x = \sigma(t_x) + c_x, \qquad b_y = \sigma(t_y) + c_y,$$
$$b_w = p_w\, e^{t_w}, \qquad b_h = p_h\, e^{t_h}.$$

The sigmoid keeps the predicted center inside its cell.

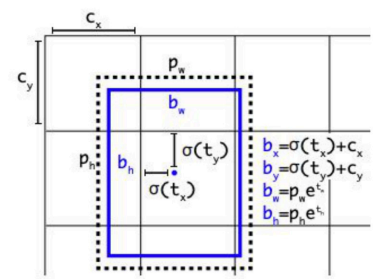
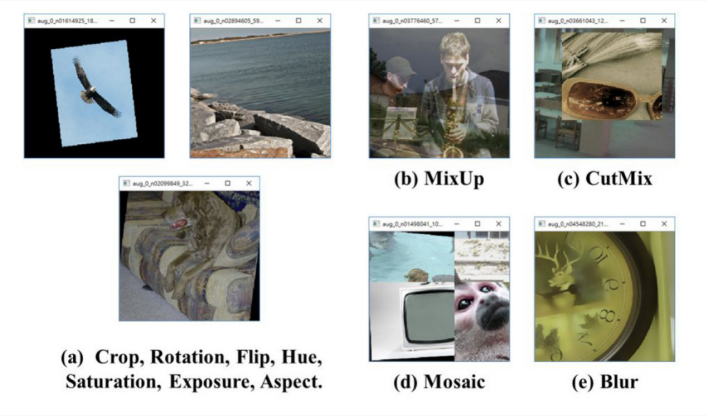


Figure 3: Bounding boxes with dimension priors and location prediction. We predict the width and height of the box as offsets from cluster centroids. We predict the center coordinates of the box relative to the location of filter application using a sigmoid function.

Redmon and Farhadi, *YOLO9000: Better, Faster, Stronger*, CVPR 2017, Figure 3.

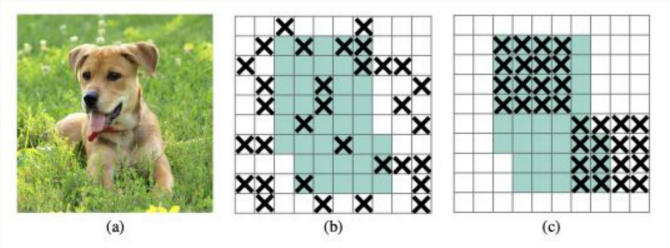
Data augmentation in YOLOv4



(a) Crop, rotation, flip, hue, saturation, exposure, aspect; (b) MixUp; (c) CutMix; (d) Mosaic; (e) blur. Bochkovskiy et al. 2020, Figure 3.

DropBlock drops contiguous regions, not single units

- DropBlock is more effective than dropout for CNNs.
- Neighboring activations are correlated, so dropping single units removes little information.
- YOLOv4 applies DropBlock as a regularizer.



Ghiasi, Lin, and Le, *DropBlock: A Regularization Method for Convolutional Networks*, NeurIPS 2018, Figure 1, arxiv.org/abs/1810.12890.

Label smoothing softens one-hot targets

Given a parameter α and K classes, the one-hot ground-truth labels y_{hot} are replaced by

$$y_{\text{ls}} = (1 - \alpha)\, y_{\text{hot}} + \frac{\alpha}{K}.$$

Example with $K = 3$

- Applying a softmax to the logits $[10, 0, 0]$ gives $[0.9999, 0, 0]$, rounded to four decimal places: the network is pushed toward extreme logits.
- With label smoothing at $\alpha = 0.1$, the target vector becomes $[0.9333,\ 0.0333,\ 0.0333]$.

After *What is label smoothing?*, Towards Data Science.

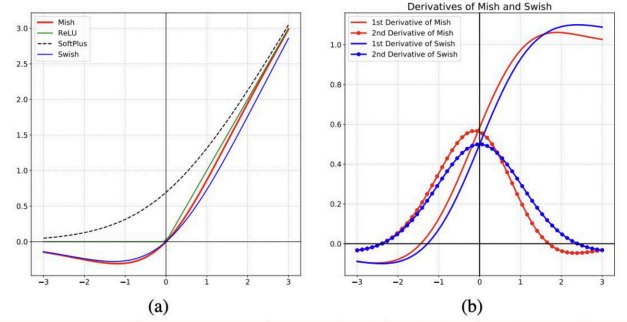
Mish: a smooth, non-monotonic activation

Mish is a non-monotonic activation function that preserves small negative values.

$$\text{mish}(x) = x \tanh\big(\text{softplus}(x)\big)$$

$$\text{softplus}(x) = \frac{1}{\beta} \log\big(1 + e^{\beta x}\big)$$

Misra, *Mish: A Self Regularized Non-Monotonic Activation Function*, BMVC 2020, arxiv.org/abs/1908.08681.



(a) Mish, ReLU, softplus, and Swish; (b) first and second derivatives of Mish and Swish. Misra 2020.

Freebies and Mish improve the classifier

MixUp	CutMix	Mosaic	Blurring	Label smoothing	Swish	Mish	Top-1	Top-5
							77.9%	94.0%
✓							77.2%	**94.0%**
	✓						**78.0%**	**94.3%**
		✓					**78.1%**	**94.5%**
			✓				77.5%	93.8%
				✓			**78.1%**	**94.4%**
					✓		64.5%	86.0%
						✓	**78.9%**	**94.5%**
	✓	✓		✓			**78.5%**	**94.8%**
	✓	✓		✓		✓	**79.8%**	**95.2%**

Influence of bag-of-freebies techniques and Mish on CSPResNeXt-50 classifier accuracy. Bochkovskiy et al. 2020, Table 2.

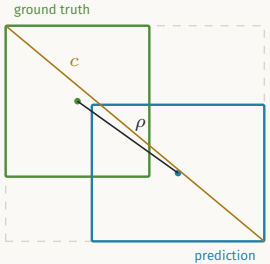
Complete IoU loss

CIoU, an extension of DIoU, considers three geometric factors: overlapping area, the distance between center points, and aspect ratio.

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2, \qquad \alpha = \frac{v}{(1-\text{IoU}) + v}$$

- $\mathbf{b}, \mathbf{b}^{gt}$: centers of the predicted and ground-truth boxes; ρ: Euclidean distance.
- c: diagonal of the smallest box enclosing both boxes; w, h: box width and height.
- v measures aspect-ratio consistency; the trade-off weight α gives overlap priority over non-overlap cases.



Formulas after Zheng et al., *Distance-IoU Loss*, AAAI 2020, and wikidocs.net/167839.

Self-adversarial training

SAT finds the part of the image the network relies on most during training and edits the image to obscure it, forcing the network to generalize to new features that help detection.

Two stages

1. The network alters the original image instead of its weights. It performs an adversarial attack on itself, changing the image to create the deception that the desired object is not there.
2. The network is then trained to detect objects on this modified image in the normal way.

Cross mini-batch normalization

CmBN collects statistics only between the mini-batches within a single batch, and updates the scale and bias parameters only at the end of the batch.

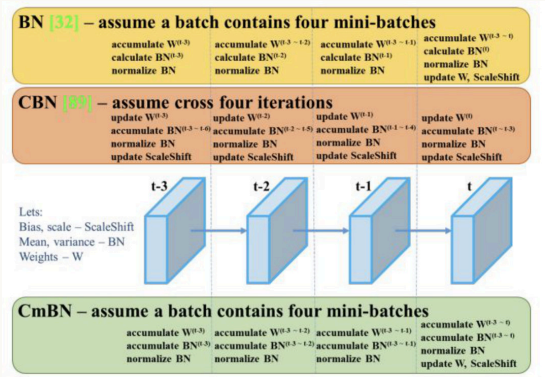
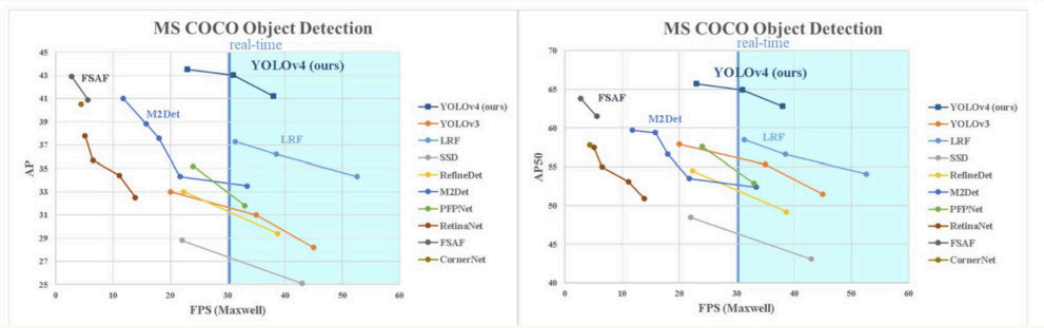


Figure 4: Cross mini-Batch Normalization.

Batch normalization (BN), cross-iteration BN (CBN), and CmBN. Bochkovskiy et al. 2020, Figure 4.

YOLOv4 sits on the speed and accuracy frontier



MS COCO AP (left) and AP_{50} (right) versus FPS on a Maxwell GPU; the shaded region is real time. Bochkovskiy et al. 2020.

The evolution of YOLO from v1 to v4

YOLOv1

Extremely fast

Each grid cell has only one class output

Poor performance on small objects

High localization error

Low recall

Trained on the 20 PASCAL VOC classes

YOLOv2

A separate class prediction for every box

About 33% less computation

YOLO9000 detects over 9000 classes

Batch norm instead of dropout, passthrough layer, anchor boxes, two-phase training

YOLOv3

Multi-label classification

Better on small objects

Larger network (53 convolutional layers), 3 boxes at 3 scales (13×13, 26×26, 52×52)

At an IoU threshold of 0.75, mAP drops significantly

YOLOv4

Several small architectural changes

At 416×416, about 38 FPS on a Maxwell GPU; AP_{50} rises from 55.3% (v3) to 62.8%

Darknet becomes CSPDarknet; several augmentations and training increments for robustness

After blog.roboflow.com/guide-to-yolo-models and the YOLOv4 paper, arxiv.org/abs/2004.10934.

PART 7

RetinaNet and focal loss

Outline

- Fundamentals of object detection
- Evaluation criteria for object detection
- Conventional object detection methods
  - HOG DPM OverFeat
- Segmentation
  - Semantic and instance segmentation
- Evaluation criteria for semantic segmentation
- Early segmentation approaches
- Fully convolutional networks (FCNs)
- U-Net segmentation
- Region-based CNNs (R-CNN)
- Fast R-CNN
- Faster R-CNN
- Multi-task network cascades
- Mask R-CNN instance segmentation
- Single Shot MultiBox Detector (SSD), a single-stage detector
- YOLO (You Only Look Once), a single-stage detector
- **RetinaNet**

Why do one-stage detectors trail in accuracy?

Object detection faces an extreme **foreground-background class imbalance**. The two families handle it differently.

Two-stage detectors

- The proposal stage quickly narrows the candidate locations to a small number (1–2k), filtering out most background samples.
- The classification stage fixes the foreground-to-background ratio at 1:3, or uses online hard example mining (OHEM).

One-stage detectors

- Must process a much larger set of candidate locations regularly sampled across the image.
- That amounts to enumerating about 100k locations that densely cover positions, scales, and aspect ratios.

Focal loss targets the imbalance directly

The diagnosis

The central cause of the lower accuracy of one-stage detectors is the extreme foreground-background class imbalance encountered while training dense detectors.

The remedy: focal loss

Reshape the standard cross-entropy loss so that it down-weights the loss assigned to well-classified examples.

Lin, Goyal, Girshick, He, and Dollár, *Focal Loss for Dense Object Detection*, ICCV 2017, arxiv.org/abs/1708.02002.

Easy background swamps the hard examples

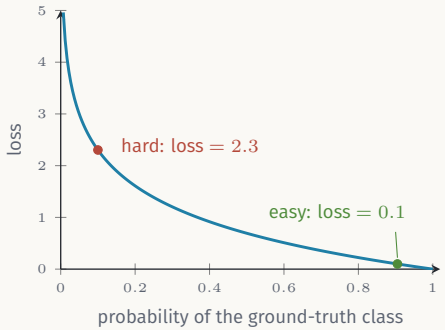
- Training a detector faces an extreme imbalance between **background**, which contains no object, and **foreground**, which holds the objects of interest.
- Focal loss gives more weight to **hard**, easily misclassified examples, such as background with noisy texture or a partial object.
- It down-weights **easy** examples, such as obviously empty background.

Start from the cross-entropy loss for binary classification:

$$\mathrm{CE}(p, y) = \begin{cases} -\log(p) & \text{if } y = 1, \\ -\log(1-p) & \text{otherwise,} \end{cases}$$

where $y \in \{\pm 1\}$ is the ground-truth label, indicating whether a box contains an object, and $p \in [0, 1]$ is the predicted probability of the class with label $y = 1$.

Cross-entropy on unbalanced data



Cross-entropy — $\log p_t$; illustrative counts.

Suppose an image yields 100,000 easy examples and 100 hard ones.

- Loss from easy (background) examples: $100{,}000 \times 0.1 = 10{,}000$.
- Loss from hard (foreground) examples: $100 \times 2.3 = 230$.
- $10{,}000/230 \approx 43$.

The result

The easy examples contribute about 40 times more loss than the hard ones, so they dominate the gradient.

One approach: α-balanced cross-entropy

For notational convenience, define

$$p_t = \begin{cases} p & \text{if } y = 1, \\ 1 - p & \text{otherwise,} \end{cases} \qquad \text{so that} \qquad \mathrm{CE}(p, y) = \mathrm{CE}(p_t) = -\log(p_t).$$

To address the class imbalance, add a weighting factor $\alpha \in [0, 1]$ for class 1 and $1 - \alpha$ for class -1, with α_t defined like p_t:

$$\mathrm{CE}(p_t) = -\alpha_t \log(p_t).$$

- α may be set by **inverse class frequency** or treated as a **hyperparameter** chosen by cross-validation.
- Two-stage detectors implement α implicitly by fixing the foreground-to-background ratio at 1:3.

The focal loss down-weights easy examples

- Easily classified examples, with $p_t \gg 0.5$ (p very close to 0 when $y = -1$, or to 1 when $y = 1$), still incur a loss of non-trivial magnitude.
- Summed over many easy examples, these small losses overwhelm the rare class.

Focal loss adds a modulating factor $(1 - p_t)^\gamma$ with $\gamma \geq 0$ to cross-entropy. The weight is small when p_t is large, so easy examples are down-weighted:

$$\mathrm{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t).$$

Adding α gives the form RetinaNet uses

For better control, add the hyperparameter α:

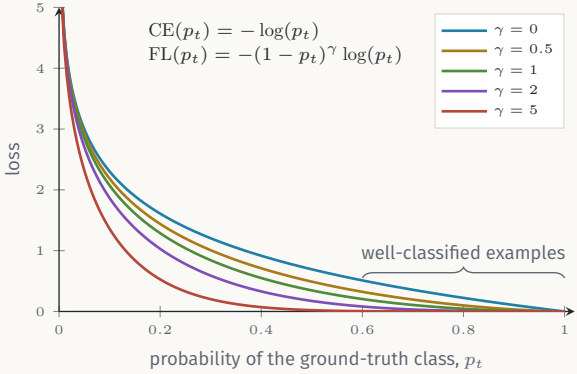
$$\mathrm{FL}(p_t) = -\alpha_t (1-p_t)^\gamma \log(p_t), \qquad \mathrm{FL}(p, y) = \begin{cases} -\alpha (1-p)^\gamma \log(p) & \text{if } y = 1, \\ -(1-\alpha)\, p^\gamma \log(1-p) & \text{otherwise.} \end{cases}$$

In RetinaNet

The α-balanced focal loss with $\alpha = 0.25$ and $\gamma = 2$ works best. α balances positive and negative examples but does not distinguish easy from hard ones; γ does.

Focal loss versus conventional cross-entropy

- Setting $\gamma > 0$ reduces the relative loss for well-classified examples ($p_t > 0.5$), putting more focus on hard, misclassified examples.
- This lets dense detectors train to high accuracy despite a vast number of easy background examples.
- $\gamma = 0$ recovers cross-entropy.



Plotted from the formula; after Lin et al., ICCV 2017, Figure 1.

Every example is weighted by how wrong it is

The focusing parameter γ smoothly adjusts the rate at which easy examples are down-weighted.

- A misclassified example has small p_t: the modulating factor is near 1 and the loss is unaffected.
- As $p_t \to 1$, the factor goes to 0 and the loss of well-classified examples is down-weighted.

With $\gamma = 2$

- At $p_t = 0.9$: $(1 - 0.9)^2 = 0.01$, so the loss is 100 times lower than cross-entropy.
- At $p_t \approx 0.968$: $(0.032)^2 \approx 0.001$, so the loss is 1000 times lower.

This raises the relative importance of correcting misclassified examples.

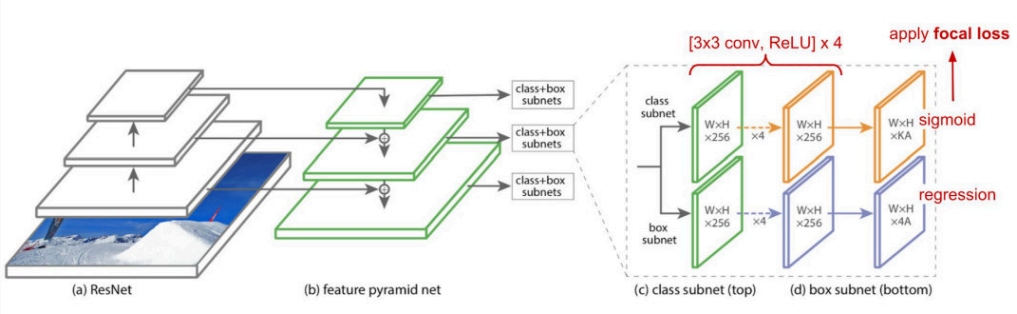
From focal loss to RetinaNet

Focal loss focuses training on a sparse set of hard examples and keeps the vast number of easy negatives from overwhelming the detector.

- To evaluate it, the authors designed a simple dense detector, **RetinaNet**.
- RetinaNet is a one-stage dense detector built on two crucial pieces: a **feature pyramid network** and the **focal loss**.
- Trained with the focal loss, RetinaNet matches the speed of previous one-stage detectors while surpassing the accuracy of all existing state-of-the-art two-stage detectors.

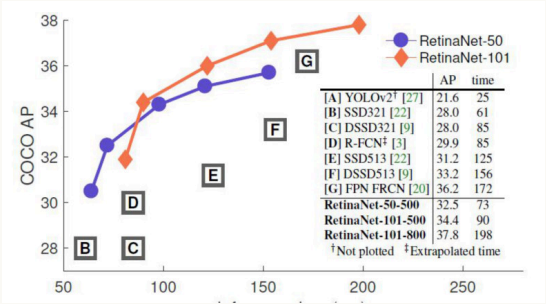
Code: github.com/facebookresearch/detectron2

The RetinaNet architecture



(a) ResNet backbone, (b) feature pyramid net, (c) class subnet, four 3×3 conv and ReLU layers followed by a sigmoid, where the focal loss is applied, (d) box subnet for regression. Lin et al., ICCV 2017, Figure 3.

RetinaNet leads on both speed and accuracy



Speed (ms) versus accuracy (AP) on COCO test-dev. Lin et al., ICCV 2017, Figure 2.

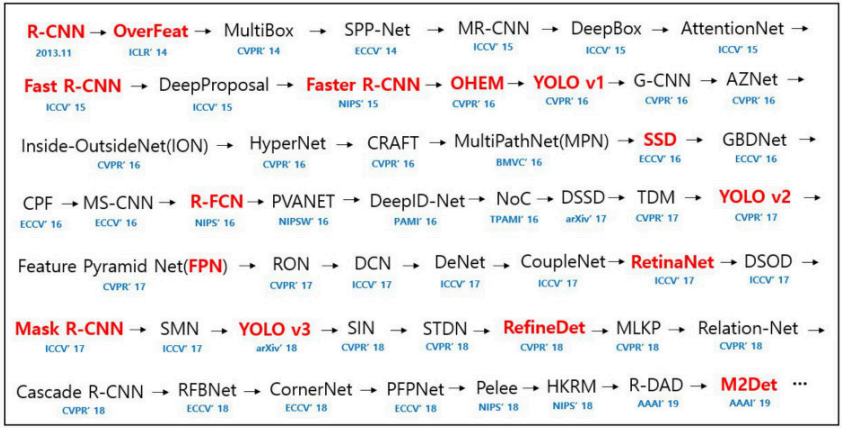
- Enabled by the focal loss, the simple one-stage RetinaNet outperforms all previous one-stage and two-stage detectors, including the best reported Faster R-CNN.
- Variants use ResNet-50-FPN (circles) and ResNet-101-FPN (diamonds) at five scales (400–800 pixels).
- Ignoring the low-accuracy regime ($AP < 25$), RetinaNet forms an upper envelope of all detectors; an improved variant (not shown) reaches 40.8 AP.

RetinaNet against the state of the art on COCO

	Backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Two-stage methods							
Faster R-CNN+++	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI	Inception-ResNet-v2	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	**52.1**
One-stage methods							
YOLOv2	DarkNet-19	21.6	44.0	19.2	5.0	22.4	35.5
SSD513	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet	ResNeXt-101-FPN	**40.8**	**61.1**	**44.1**	**24.1**	**44.2**	51.2

Single-model bounding box AP on COCO test-dev. The RetinaNet-101-800 model is trained with scale jitter and 1.5× longer than in the ablations. Lin et al., ICCV 2017, Table 2.

A snapshot of object detection methods up to 2019



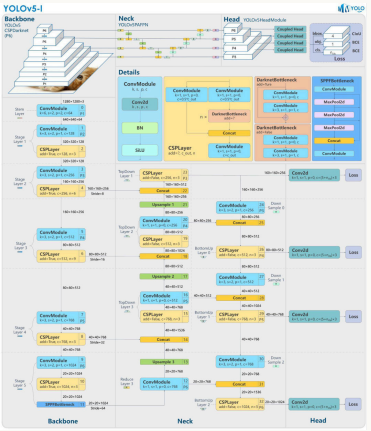
github.com/hoya012/deep_learning_object_detection.

PART 8

YOLOv5 to YOLOv8

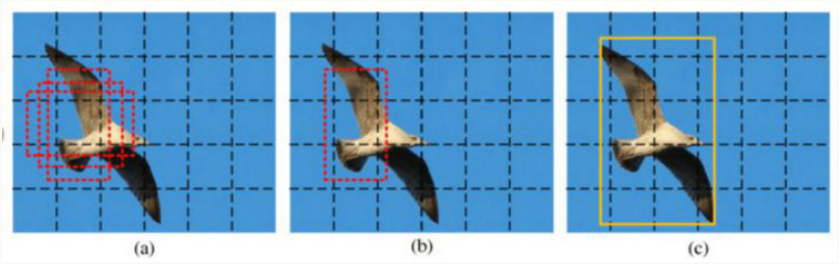
YOLOv5: backbone, neck, and head

- The YOLOv5 architecture in full: a CSP backbone, a PAN neck, and a three-scale head.
- The following slides look at its parts: adaptive anchors, the Focus layer, and the C3 and Bottleneck modules.



YOLOv5-l, MMYOLO.

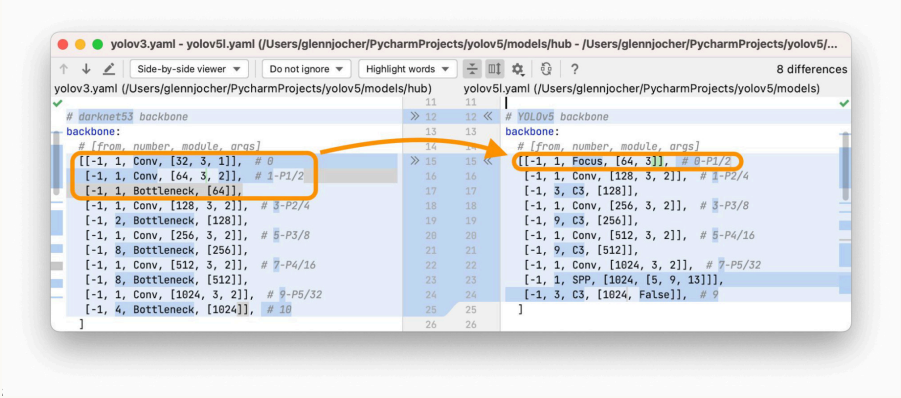
YOLOv5 computes anchors adaptively



Adaptive anchor boxes

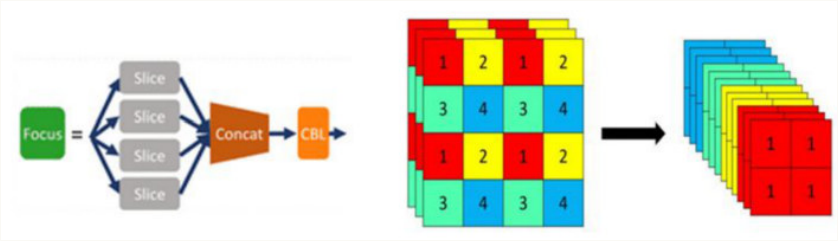
The best anchor box sizes differ between training sets, so YOLOv5 calculates them adaptively from the data at the start of each training run.

Focus replaces the first three YOLOv3 layers



Side-by-side comparison of the YOLOv3 (left) and YOLOv5 (right) model configurations in the Ultralytics repository.

Focus slices the image into four smaller copies



- The Focus structure takes every second pixel in each direction, splitting a high-resolution image into four low-resolution images.
- Stacking them moves spatial information into channels, $(b, c, w, h) \rightarrow (b, 4c, w/2, h/2)$, before a single convolution.

The Focus layer in code

```
class Focus(nn.Module):
    # Focus wh information into c-space
    # ch_in, ch_out, kernel, stride, padding, groups
    def __init__(self, c1, c2, k=1, s=1, p=None, g=1, act=True):
        super(Focus, self).__init__()
        self.conv = Conv(c1 * 4, c2, k, s, p, g, act)
        # self.contract = Contract(gain=2)

    def forward(self, x):  # x(b,c,w,h) -> y(b,4c,w/2,h/2)
        return self.conv(torch.cat([x[..., ::2, ::2], x[..., 1::2, ::2],
                                    x[..., ::2, 1::2], x[..., 1::2, 1::2]], 1))
```

- The four slices take the even and odd rows and columns; `torch.cat` stacks them along the channel dimension.

C3: a CSP bottleneck with three convolutions

```python
class C3(nn.Module):
    # CSP Bottleneck with 3 convolutions
    # ch_in, ch_out, number, shortcut, groups, expansion
    def __init__(self, c1, c2, n=1, shortcut=True, g=1, e=0.5):
        super().__init__()
        c_ = int(c2 * e)  # hidden channels
        self.cv1 = Conv(c1, c_, 1, 1)
        self.cv2 = Conv(c1, c_, 1, 1)
        self.cv3 = Conv(2 * c_, c2, 1)  # optional act=FReLU(c2)
        self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, e=1.0) for _ in range(n)))

    def forward(self, x):
        return self.cv3(torch.cat((self.m(self.cv1(x)), self.cv2(x)), 1))
```

- `cv1` feeds a stack of n bottlenecks; `cv2` is the CSP bypass; `cv3` fuses the concatenation.

github.com/ultralytics/yolov5, models/common.py.

The Bottleneck module

```python
class Bottleneck(nn.Module):
    # Standard bottleneck
    # ch_in, ch_out, shortcut, groups, kernels, expand
    def __init__(self, c1, c2, shortcut=True, g=1, k=(3, 3), e=0.5):
        super().__init__()
        c_ = int(c2 * e)  # hidden channels
        self.cv1 = Conv(c1, c_, k[0], 1)
        self.cv2 = Conv(c_, c2, k[1], 1, g=g)
        self.add = shortcut and c1 == c2

    def forward(self, x):
        return x + self.cv2(self.cv1(x)) if self.add else self.cv2(self.cv1(x))
```

- Two convolutions with a residual connection, used only when the input and output channel counts match.

github.com/ultralytics/yolov5, models/common.py.

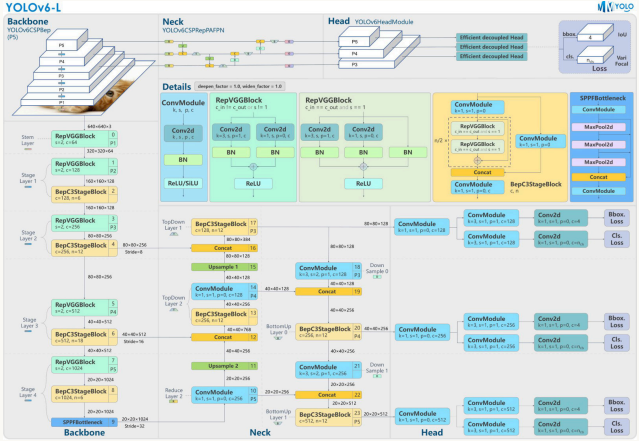
YOLOv5 can train with focal loss

An optional loss

YOLOv5 optionally allows training with the focal loss from the RetinaNet section:

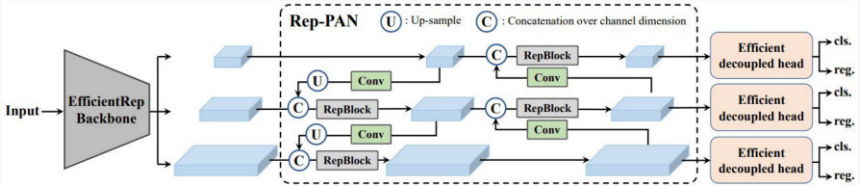
$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t).$$

YOLOv6



YOLOv6-l, MMYOLO.

What changed in YOLOv6



EfficientRep backbone, Rep-PAN neck, and efficient decoupled heads. Li et al., *YOLOv6*, arXiv 2022.

- **Backbone**: RepVGG-style multi-branch blocks are used during training and replaced by simple 3×3 convolutions at inference.
- **Neck**: a PAN that concatenates features from re-parameterized blocks, hence called Rep-PAN.
- **Head**: an efficient decoupled head, in which the classification and box regression branches do not share parameters.
- **Classification loss**: varifocal loss.

RepVGG: train with branches, infer without

- During training, each block has a 3×3 convolution plus 1×1 and identity branches, like a ResNet.
- After training, the branches are merged algebraically into a single 3×3 convolution.
- Inference runs on a plain VGG-like stack, which is fast on GPUs.

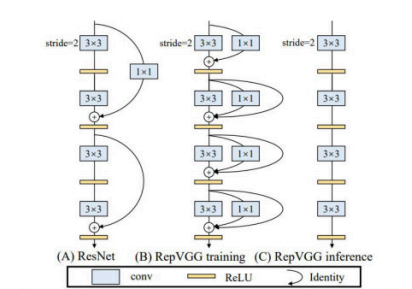


Figure 2: Sketch of RepVGG architecture. RepVGG has 5 stages and conducts down-sampling via stride-2 convolution at the beginning of a stage. Here we only show the first 4 layers of a specific stage. As inspired by ResNet [12], we also use identity and 1×1 branches, but only for training.

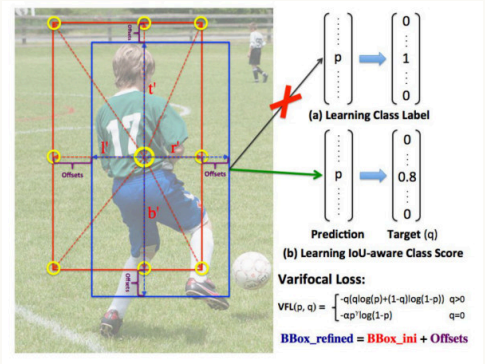
Ding et al., *RepVGG*, CVPR 2021, arxiv.org/abs/2101.03697.

Varifocal loss learns an IoU-aware class score

Instead of a one-hot class label, the target q for a positive example is the IoU between the predicted box and the ground truth; for negatives, $q = 0$:

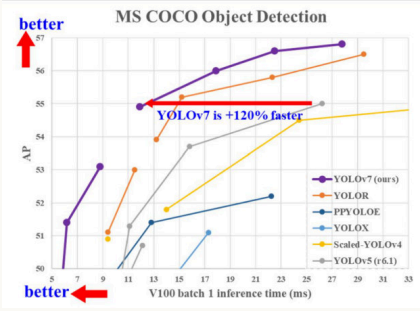
$$\mathrm{VFL}(p, q) = \begin{cases} -q\big(q \log p + (1-q)\log(1-p)\big), & q > 0 \\ -\alpha\, p^{\gamma} \log(1-p), & q = 0 \end{cases}$$

- Only negatives are down-weighted, as in focal loss.
- Positives are weighted by their own IoU, so high-quality boxes matter most.

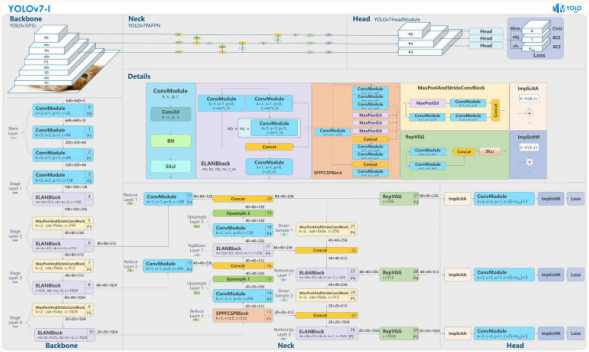


Zhang, Wang, Dayoub, and Sünderhauf, *VarifocalNet: An IoU-aware Dense Object Detector*, CVPR 2021.

YOLOv7



MS COCO AP versus V100 inference time. Wang, Bochkovskiy, and Liao, *YOLOv7*, CVPR 2023.



YOLOv7-l, MMYOLO.

Efficient layer aggregation networks

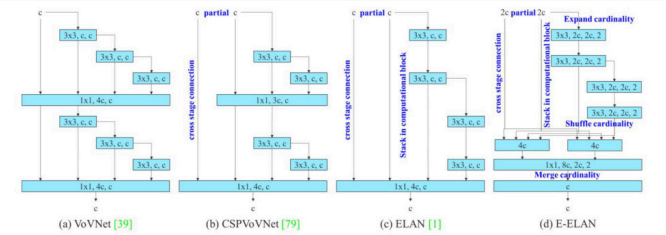


Figure 2: Extended efficient layer aggregation networks. The proposed extended ELAN (E-ELAN) does not change the gradient transmission path of the original architecture at all, but use group convolution to increase the cardinality of the added features, and combine the features of different groups in a shuffle and merge cardinality manner. This way of operation can enhance the features learned by different feature maps and improve the use of parameters and calculations.

- Extended ELAN keeps the gradient path of the original architecture, uses group convolution to increase the cardinality of the added features, and combines the features of different groups by shuffling and merging cardinality.

Wang et al., *YOLOv7*, Figure 2. See also github.com/WongKinYiu/yolov7/issues/762.

Model scaling for concatenation-based models

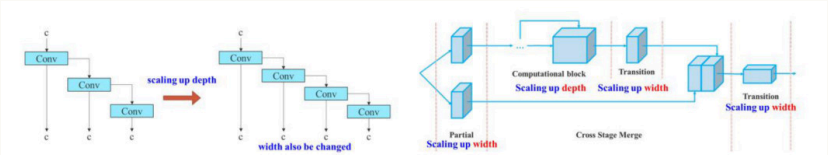


Figure 3: Model scaling for concatenation-based models. From (a) to (b), we observe that when depth scaling is performed on concatenation-based models, the output width of a computational block also increases. This phenomenon will cause the input width of the subsequent transmission layer to increase. Therefore, we propose (c), that is, when performing model scaling on concatenation-based models, only the depth in a computational block needs to be scaled, and the remaining of transmission layer is performed with corresponding width scaling.

- Scaling up the depth of a concatenation-based block also widens its output, which changes the input width of the next transmission layer.
- YOLOv7 therefore scales the depth of the computational block and scales the width of the remaining transmission layer to match.

Wang et al., *YOLOv7*, Figure 3.

Deep supervision with lead and auxiliary heads

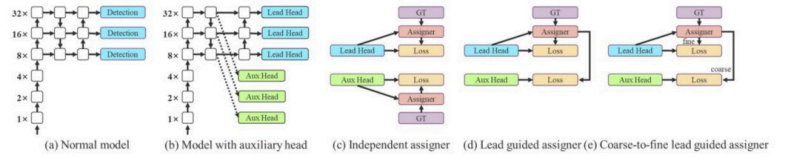


Figure 5: Coarse for auxiliary and fine for lead head label assigner. Compare with normal model (a), the schema in (b) has auxiliary head. Different from the usual independent label assigner (c), we propose (d) lead head guided label assigner and (e) coarse-to-fine lead head guided label assigner. The proposed label assigner is optimized by lead head prediction and the ground truth to get the labels of training lead head and auxiliary head at the same time. The detailed coarse-to-fine implementation method and constraint design details will be elaborated in Apendix.

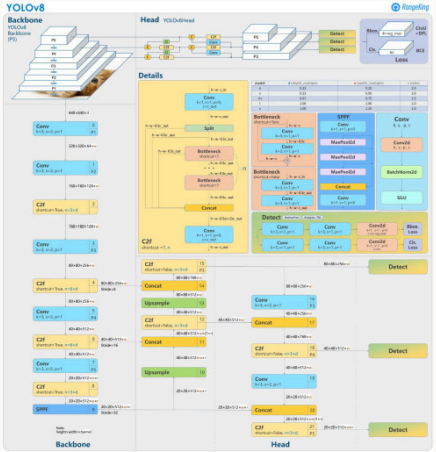
- An auxiliary head supervises intermediate layers during training, in addition to the lead head.
- In the lead-head-guided assigner, the lead head's predictions and the ground truth together produce the training labels for both heads; the coarse-to-fine variant gives the auxiliary head coarser labels.

Wang et al., *YOLOv7*, Figure 5.

YOLOv8

Compared with YOLOv5

- The C3 module (a three-convolution CSP block) is replaced by C2f.
- The first 6×6 convolution in the backbone becomes 3×3.
- Two convolutions (No. 10 and No. 14 in the YOLOv5 config) are removed.
- The first 1×1 convolution in the Bottleneck becomes 3×3.
- The head is decoupled and the objectness branch is removed.



YOLOv8, MMYOLO.

C2f replaces C3 in YOLOv8

```python
class C2f(nn.Module):
    # CSP Bottleneck with 2 convolutions
    def __init__(self, c1, c2, n=1, shortcut=False, g=1, e=0.5):  # ch_in, ch_out, number, shortcut, groups, expansion
        super().__init__()
        self.c = int(c2 * e)  # hidden channels
        self.cv1 = Conv(c1, 2 * self.c, 1, 1)
        self.cv2 = Conv((2 + n) * self.c, c2, 1)  # optional act=FReLU(c2)
        self.m = nn.ModuleList(Bottleneck(self.c, self.c, shortcut, g, k=((3, 3), (3, 3)), e=1.0) for _ in range(n))

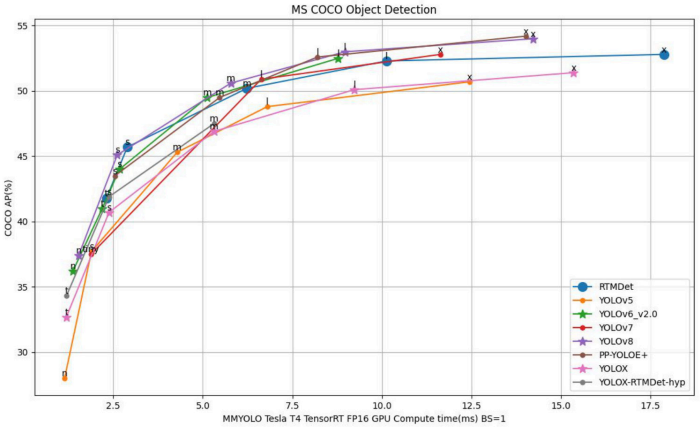
    def forward(self, x):
        y = list(self.cv1(x).chunk(2, 1))
        y.extend(m(y[-1]) for m in self.m)
        return self.cv2(torch.cat(y, 1))

    def forward_split(self, x):
        y = list(self.cv1(x).split((self.c, self.c), 1))
        y.extend(m(y[-1]) for m in self.m)
        return self.cv2(torch.cat(y, 1))
```

- C2f concatenates every bottleneck's output; C3 (YOLOv5 section) concatenates only the last one with its bypass.

github.com/ultralytics/ultralytics, ultralytics/nn/modules.py.

YOLOv8 against other real-time detectors



COCO AP versus Tesla T4 TensorRT FP16 compute time (batch size 1), github.com/open-mmlab/mmyolo.

Questions?

Mehmet Kerem Turkcan Columbia University